Predicting Layoff Risk in the Automation Era: A Four-Model Comparative Analysis Using the AI Impact on Jobs Dataset

Mustafa Zendaki and Saman Zarei

Group 7

M.S. in Applied Artificial Intelligence, University of San Diego

AAI-500: Applied Probability and Statistics for AI

Professor Leon Shpaner

October 10, 2026

Abstract

We evaluated whether occupational and workplace automation characteristics could predict assigned layoff-risk categories in 20,000 synthetic employee records. Five predictors represented industry, years of experience, routine tasks, task automation, and AI tool use. After quality checks, encoding, and standardization, we used a stratified 80%–20% training and test split to compare multinomial logistic regression, random forest, gradient boosting, and Gaussian naïve Bayes. Evaluation included accuracy, precision, recall, specificity, F1 scores, and confusion matrices. Logistic regression achieved the highest test accuracy (75.025%) and macro F1 (0.7492), followed in accuracy by gradient boosting (74.950%), Gaussian naïve Bayes (73.350%), and random forest (72.625%). Gradient boosting achieved the highest High-risk recall (0.8353), whereas logistic regression achieved the highest High-risk precision and specificity. Medium risk was the most difficult category for all four models. We selected logistic regression as the preferred overall model for its observed performance and interpretability, with gradient boosting as an alternative when High-risk detection is prioritized. Their accuracy difference represented only three additional correct classifications, and statistical superiority was not established. These findings concern synthetic risk labels; repeated validation and evaluation using observed layoff outcomes are needed before practical application.

Keywords: layoff risk, workplace automation, multiclass classification, machine learning

Table of Contents

Abstract 2

Introduction 4

Data Cleaning and Preparation 4

Exploratory Data Analysis 5

AI Tool Use and Industry Patterns 7

Correlations and Multicollinearity 10

EDA Summary 11

Model Selection 11

Rationale for the Four Selected Models 12

Shared Settings and Evaluation 13

Model Analysis 14

Multinomial Logistic Regression 15

Random Forest 18

Gradient Boosting 20

Gaussian Naïve Bayes 22

Four-Model Comparison 24

Limitations 25

Conclusion and Recommendations 26

References 27

GitHub 28

Appendix 29

Introduction

Research on computerization and industrial robots links technological change to shifts in job tasks and employment (Acemoglu & Restrepo, 2020; Autor et al., 2003). These findings motivate questions about how workplace automation relates to layoff risk. We investigated whether industry, years of experience, and automation exposure could distinguish the Low, Medium, and High categories in the AI Impact on Jobs and Layoff Risk Dataset (Singh, 2026). This dataset contains 20,000 synthetic employee records. Our objective was to compare four classification models and examine both overall performance and the ability to identify High-risk observations.

We selected industry, years of experience, routine task percentage, tasks automated percentage, and number of AI tools used as predictors. We compared multinomial logistic regression, random forest, gradient boosting, and Gaussian naïve Bayes using the same training and test observations. Because the labels are synthetic, we interpret the results as patterns within this dataset rather than evidence of causal effects or validated predictions of actual layoffs.

Data Cleaning and Preparation

We checked the 16-column dataset, containing 15 candidate predictors and the target, for missing values, duplicate rows, inconsistent category labels, and implausible values. No missing values, duplicates, or label corrections were identified, so we retained all observations without imputation. All records passed our project-specific check that age minus years of experience was at least 16. We did not remove or cap outliers.

The selected numerical predictors also met their expected bounds: experience ranged from 0 to 32 years, routine tasks from 10% to 94%, automated tasks from 4% to 93%, and AI tool counts from 0 to 10. We manually selected the five predictors to represent professional years

of experience, industry, and automation exposure; no automated feature-selection procedure was used.

We encoded industry into seven binary indicators, using information technology (IT) as the reference category. This produced 11 input columns: four numerical predictors and seven industry indicators. We coded Low, Medium, and High as 0, 1, and 2, respectively, and treated them as multiclass labels rather than fitting an ordinal model.

Using a random seed of 42, we created a stratified split with 16,000 training records and 4,000 test records. The training counts were 5,282 Low, 5,281 Medium, and 5,437 High; test counts were 1,320, 1,320, and 1,360, respectively. The categories were approximately balanced, so we did not resample them. No separate validation set was created.

We fitted StandardScaler only on the training data and applied those parameters to the test data. For each numerical predictor, we subtracted its training mean and divided by its training standard deviation, preventing test information from entering the scaling estimates. The numerical inputs were standardized, while the industry indicators remained binary. We exported four CSV files containing the training and test predictors and their corresponding labels. Final checks confirmed matching schemas and row counts and no missing values. Exploratory Data Analysis (EDA) used only the training partition, with numerical values returned to their original units.

Exploratory Data Analysis

We examined class balance, numerical distributions, industry patterns, and correlations. Table 1 summarizes the numerical predictors by risk category, and Figure 1 shows their distributions. Routine tasks, automated tasks, and AI tool use generally increased across the risk

categories. Experience showed smaller differences and substantial overlap, suggesting that no single predictor fully separated the classes.

Table 1

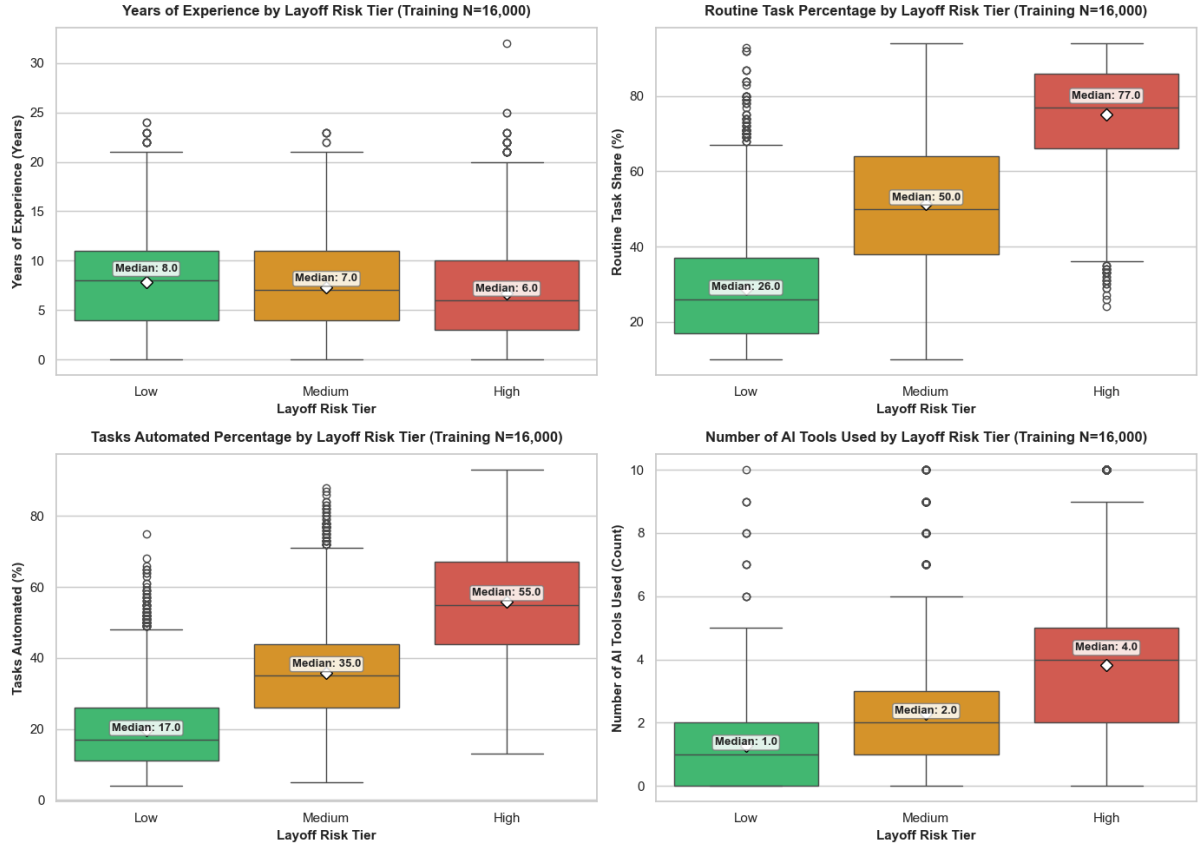
Numerical Predictor Summaries by Layoff Risk

Predictor	Low	Medium	High
Years of experience	7.85 (4.73)	7.29 (4.62)	6.68 (4.76)
Routine task share (%)	28.74 (14.54)	51.26 (17.70)	75.11 (13.45)
Tasks automated (%)	19.56 (10.61)	35.73 (13.78)	55.98 (15.10)
Number of AI tools used	1.26 (1.19)	2.27 (1.90)	3.82 (2.21)

Note. Values are *M* (*SD*) in original units. Training n = 5,282 Low, 5,281 Medium, and 5,437 High.

Figure 1

Numerical Predictor Distributions by Layoff Risk



Note. N = 16,000. Boxes show the middle 50%; horizontal lines mark medians and diamonds mark means. Whiskers extend to 1.5 interquartile ranges; points beyond them were retained.

Routine-task medians rose from 26% to 50% to 77% across Low, Medium, and High risk; automated-task medians rose from 17% to 35% to 55%. Experience medians declined from 8 to 7 to 6 years, while AI tool-count medians increased from 1 to 2 to 4. The overlapping distributions were especially relevant to distinguishing Medium risk.

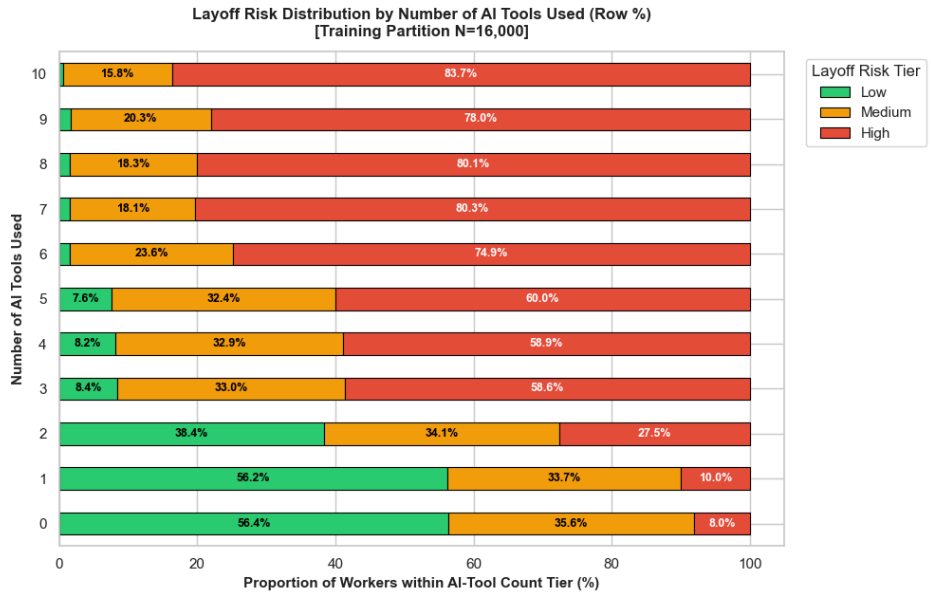
AI Tool Use and Industry Patterns

Figure 2 shows that the High-risk share rose from 8.05% among records with no AI tools to 58.61% among those with three tools. Shares for 6 to 10 tools ranged from 74.87% to 83.68%,

but these groups were much smaller. This was a descriptive association, not evidence that AI tool use caused higher risk.

Figure 2

Layoff Risk Distribution Within AI Tool Count Groups



Note. N = 16,000. Bars total 100% within each tool-count group. Group sizes for 0–10 tools were 2,834; 2,857; 4,328; 1,614; 1,595; 1,835; 191; 188; 186; 182; and 190. Labels below 5% are omitted.

Industry profiles also differed (Table 2 and Figure 3). Manufacturing had the largest High-risk share, while Education and Healthcare had the smallest. IT was our modeling reference category. We retained industry to examine whether these differences remained informative alongside the numerical predictors.

Table 2

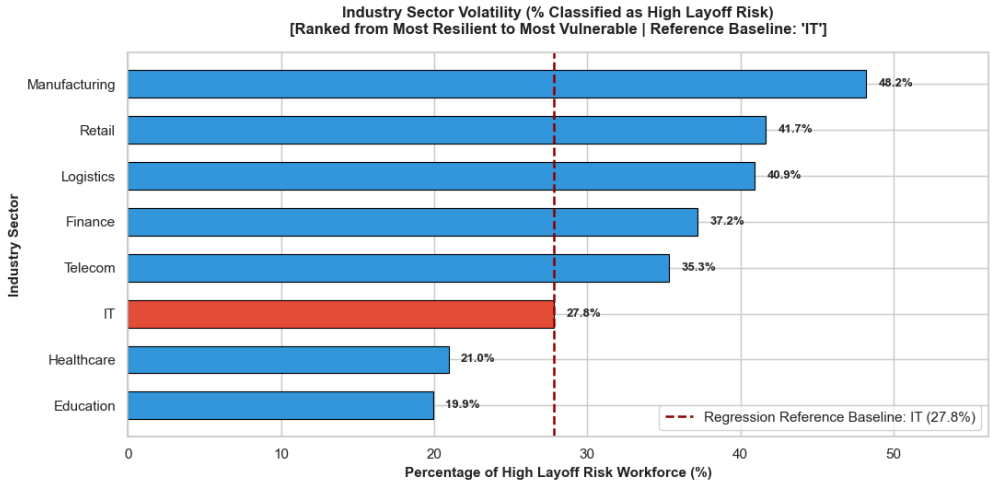
Layoff Risk Profiles Within Industries

Industry	*n*	Low (%)	Medium (%)	High (%)
Manufacturing	1,971	19.74	32.06	48.20
Retail	1,973	24.48	33.86	41.66
Logistics	1,991	25.01	34.05	40.93
Finance	2,007	27.35	35.43	37.22
Telecom	2,071	32.98	31.68	35.35
IT	1,992	38.76	33.43	27.81
Healthcare	2,038	46.76	32.29	20.95
Education	1,957	48.80	31.27	19.93

Note. N = 16,000. Percentages use each industry's sample size and may not sum to 100% after rounding. IT = information technology.

Figure 3

Percentage of High-Risk Records Within Each Industry



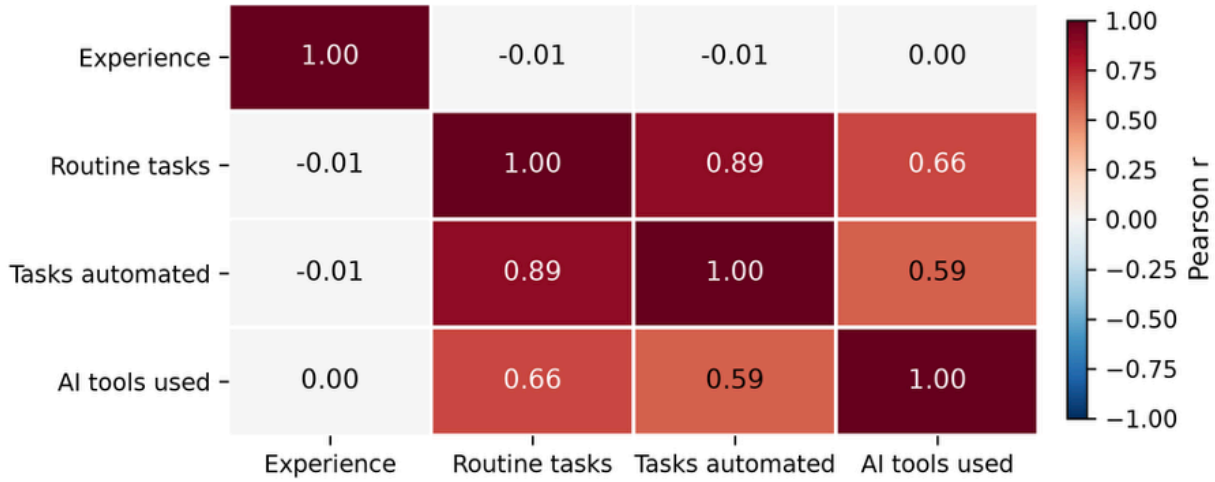
Note. Percentages are within industries; the dashed line marks the IT reference.

Correlations and Multicollinearity

Routine task percentage and tasks automated percentage were strongly correlated (r = 0.890). AI tool count correlated with routine tasks (r = 0.663) and automated tasks (r = 0.589), while experience had near-zero linear correlations with the other numerical predictors (Figure 4).

Figure 4

Pearson Correlations Among Numerical Predictors



Note. N = 16,000. Values are rounded to two decimal places.

Variance inflation factors were 5.591 for routine tasks, 4.794 for automated tasks, 1.786 for AI tools, and 1.00 for experience. These calculations included an intercept and excluded industry indicators. We retained the predictors, recognizing that collinearity can make separate coefficient estimates difficult to interpret (Dormann et al., 2013).

EDA Summary

Overall, the training data showed approximately balanced classes and increasing automation measures across risk categories. Industry patterns differed, while experience showed substantial overlap. These findings motivated comparing additive and flexible models, but did not establish which would perform best. They describe associations in synthetic records rather than causes of actual layoffs.

Model Selection

We considered six models and selected four based on our familiarity with them and their different assumptions. We chose multinomial logistic regression, random forest, gradient boosting, and Gaussian naïve Bayes. We excluded k-nearest neighbors (KNN) because distance and feature weighting required additional choices, and a single decision tree because ensembles already tested flexible rules. These scope decisions do not imply poorer performance by the excluded models.

We compared accuracy, precision, recall, specificity, F1, and confusion matrices. Accuracy measures the proportion classified correctly; precision measures the reliability of a predicted class; recall measures class detection; and specificity measures exclusion of other classes. F1 combines precision and recall through their harmonic mean (Rashidi et al., 2023). Macro averages weight classes equally, while weighted averages use class counts. Specificity was calculated for each class versus the other two classes.

Rationale for the Four Selected Models

We selected multinomial logistic regression to test whether the increases in routine and automated task shares across risk categories could be represented through additive predictor effects. It also allowed us to compare industries with IT while adjusting for experience and automation exposure. As a probability-based classification model, logistic regression provided an interpretable baseline (Rashidi et al., 2023).

We selected random forest to investigate thresholds and interactions suggested by the industry differences and overlapping numerical distributions. For example, a given automation percentage might be informative in combination with industry or experience. Combining multiple trees allowed us to examine these patterns without specifying each interaction beforehand, although EDA did not establish that this flexibility would improve prediction.

We selected gradient boosting because Medium-risk profiles overlapped with both other categories. Its sequential addition of trees to reduce classification loss offered a way to refine predictions for these overlapping combinations of task composition, experience, and industry (Friedman, 2001). We tested this possibility rather than assuming it would resolve the overlap.

We selected Gaussian naïve Bayes to assess how much information the differences in class-specific feature distributions could capture. The model uses feature means and variances within each class under Gaussian and conditional-independence assumptions. Binary industry indicators and correlated automation measures made these assumptions important limitations to examine, so we treated it as a simplified probabilistic baseline.

Shared Settings and Evaluation

Table 3

Explicitly Specified Model Settings

Model	Settings supplied in the code
Multinomial logistic regression	Maximum iterations = 1,000; random state = 42
Random forest	Number of trees = 100; random state = 42
Gradient boosting	Random state = 42
Gaussian naïve Bayes	No arguments supplied; default configuration

Note. Other settings retained the library defaults. All models used the same 11 inputs and split, without hyperparameter search or cross-validation.

Model Analysis

Table 4 consolidates the test metrics. All four models exceeded the 34.00% majority-class reference. Figures 5–12 show their error patterns and interpretation outputs.

Table 4

Test Performance Across the Four Models

Metric	LR	GB	GNB	RF
Overall performance				
Correct classifications (*n*)	3,001	2,998	2,934	2,905
Total misclassifications (*n*)	999	1,002	1,066	1,095
Test accuracy (%)	75.025	74.950	73.350	72.625
Macro precision	0.7488	0.7478	0.7264	0.7248
Macro recall (sensitivity)	0.7496	0.7486	0.7328	0.7254
Macro specificity	0.8753	0.8748	0.8669	0.8632
Macro F1	0.7492	0.7480	0.7272	0.7250
Weighted precision	0.7495	0.7483	0.7272	0.7254
Weighted recall	0.7502	0.7495	0.7335	0.7262
Weighted F1	0.7498	0.7486	0.7280	0.7257
Low risk (*n* = 1,320)				
Precision	0.8084	0.8205	0.7550	0.8012
Recall	0.8212	0.8000	0.8659	0.7848
Specificity	0.9041	0.9138	0.8616	0.9041
F1	0.8147	0.8101	0.8066	0.7930
Medium risk (*n* = 1,320)				
Precision	0.6255	0.6267	0.6187	0.5924
Recall	0.6174	0.6106	0.5311	0.5826
Specificity	0.8179	0.8209	0.8388	0.8026
F1	0.6214	0.6186	0.5715	0.5875
High risk (*n* = 1,360)				
Precision	0.8127	0.7961	0.8056	0.7807
Recall	0.8103	0.8353	0.8015	0.8088
Specificity	0.9038	0.8898	0.9004	0.8830
F1	0.8115	0.8152	0.8035	0.7945

Note. N = 4,000. LR = multinomial logistic regression; GB = gradient boosting; GNB = Gaussian naïve Bayes; RF = random forest. Class counts appear in the group labels. Specificity uses one versus rest comparisons; weighted specificity was not calculated. Values are rounded to four decimals, or three for accuracy percentages.

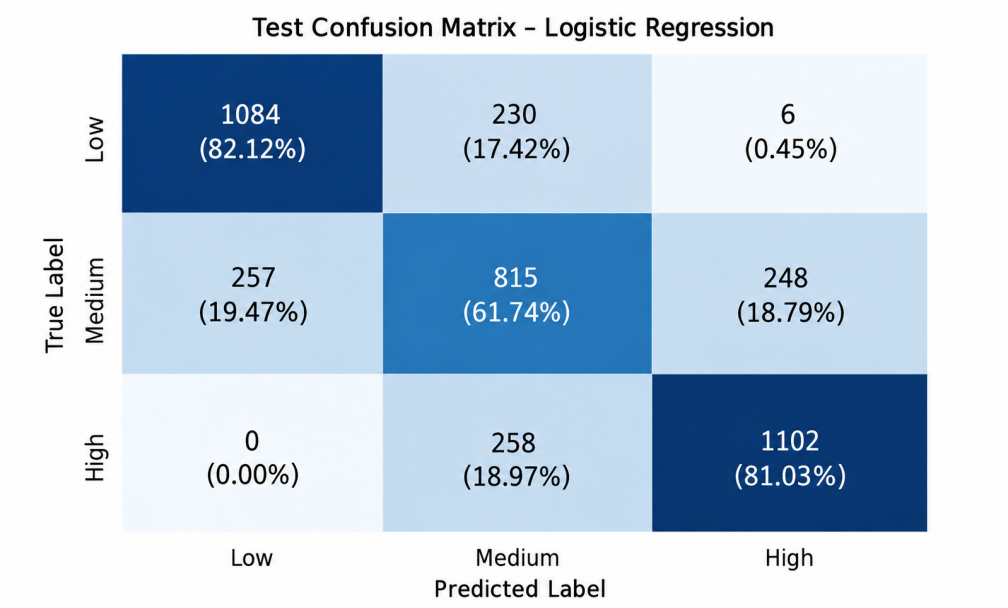
Multinomial Logistic Regression

The fitted model had three sets of 11 coefficients and used a softmax transformation to obtain class probabilities. We assigned each record to the class with the highest predicted probability.

Logistic regression correctly classified 3,001 records and achieved a macro one versus rest ROC-AUC of 0.9030. AUC summarizes probability ranking across class versus rest problems; it is not classification accuracy or a measure of calibration. Its main difficulty was Medium risk: 505 of 1,320 Medium records were misclassified. Among true High-risk records, 258 were assigned Medium and none were assigned Low (Figure 5). High-risk specificity of 0.9038 means that 90.38% of Low and Medium-risk records were correctly excluded from the High category.

Figure 5

Logistic Regression Confusion Matrix on the Test Set



Note. Rows show recorded classes and columns show predictions. Cells contain counts and row percentages. $N = 4{,}000$.

To interpret the multinomial model, we calculated pairwise odds ratios by exponentiating differences between class coefficients. For High versus Low, the ratio is $\exp(\beta_{\text{High}} - \beta_{\text{Low}})$, holding other inputs constant. Table 5 reports all three contrasts, and Figure 6 displays High versus Low. Numerical ratios refer to one training standard deviation: 4.73 years of experience, 24.39 percentage points of routine tasks, 20.01 percentage points of automation, or 2.10 AI tools.

Table 5

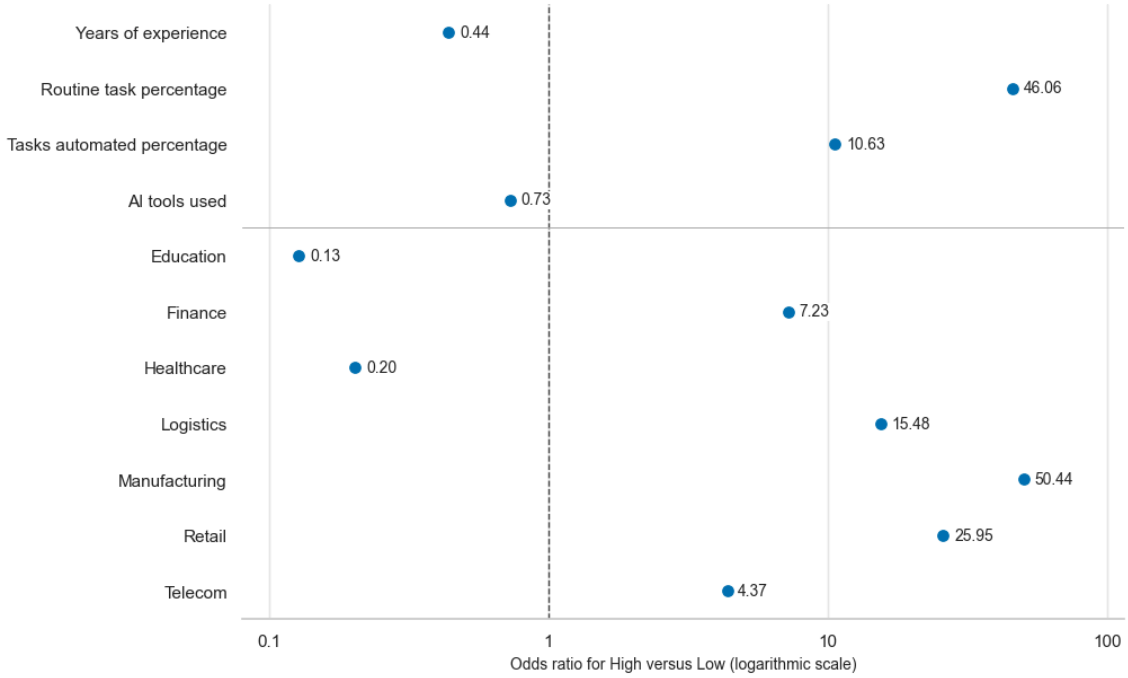
Pairwise Odds Ratios for Logistic Regression

Predictor	Medium vs. Low	High vs. Low	High vs. Medium
Years of experience	0.67	0.44	0.65
Routine task percentage	6.26	46.06	7.36
Tasks automated percentage	3.40	10.63	3.12
AI tools used	0.86	0.73	0.84
Education	0.34	0.13	0.37
Finance	2.88	7.23	2.51
Healthcare	0.45	0.20	0.45
Logistics	4.05	15.48	3.82
Manufacturing	7.43	50.44	6.79
Retail	5.39	25.95	4.81
Telecom	1.99	4.37	2.20

Note. Ratios are rounded to two decimals. Numerical predictors use one-standard-deviation increases; industry indicators compare each sector with IT. Contrasts describe relative class probabilities, not changes in absolute layoff probability. No confidence intervals were calculated.

Figure 6

Logistic Regression Odds Ratios for High Versus Low Risk



Note. The dashed line marks an odds ratio of 1. Numerical predictors use one standard deviation increases; industry comparisons use IT as the reference. No uncertainty intervals are shown.

Routine tasks and automated tasks had High versus Low odds ratios of approximately 46.06 and 10.63. Experience and AI tools had ratios below 1. The adjusted AI-tool association differed from its positive association in EDA, illustrating why marginal patterns and conditional effects should not be treated as equivalent. Manufacturing had the largest industry ratio (50.44), while Education and Healthcare had ratios of 0.13 and 0.20. Correlated predictors limit interpretation of separate coefficients; these associations do not establish causal effects.

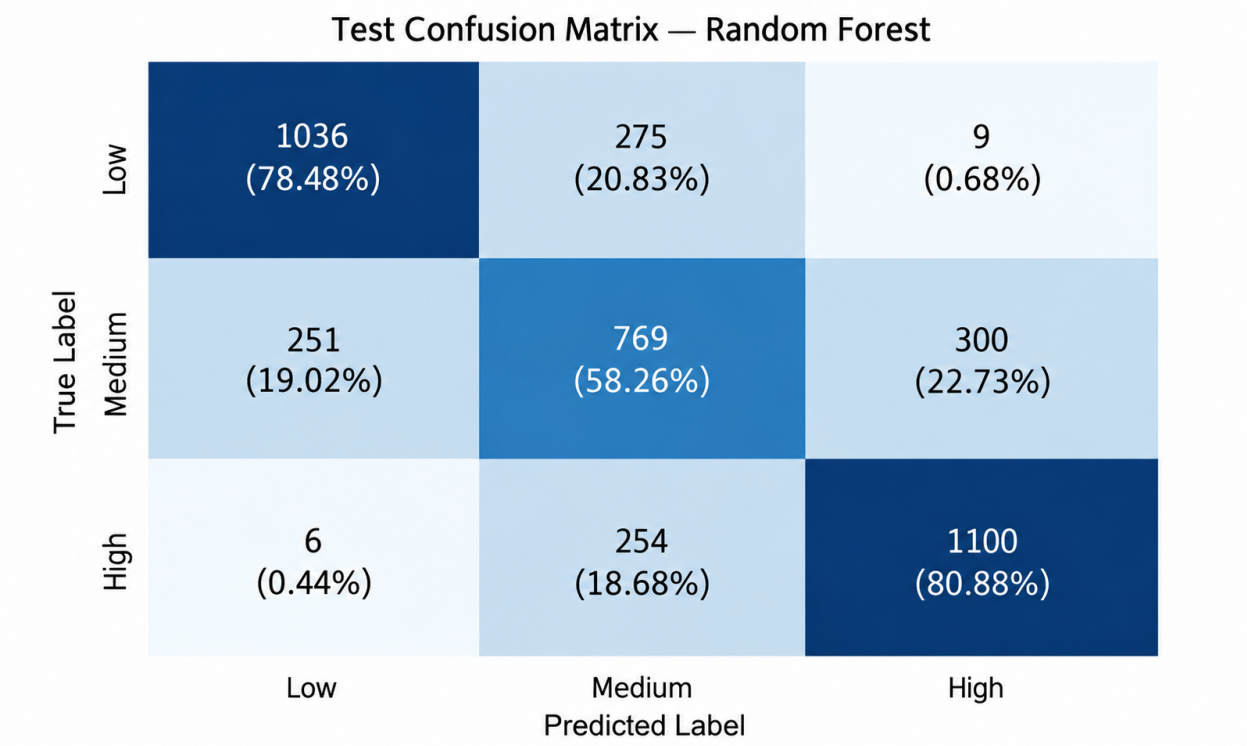
Random Forest

We fitted 100 trees and assigned the class with the highest average class probability across them. The 11 input columns describe the predictors available to the forest, rather than the number considered at each split.

Random forest correctly classified 2,905 records and achieved a macro ROC-AUC of 0.8867. Figure 7 shows 551 errors among true Medium-risk observations. It detected 1,100 High-risk observations but produced 309 false High-risk predictions. Its High-risk specificity was 0.8830, indicating that 88.30% of non-High observations were correctly excluded from that category.

Figure 7

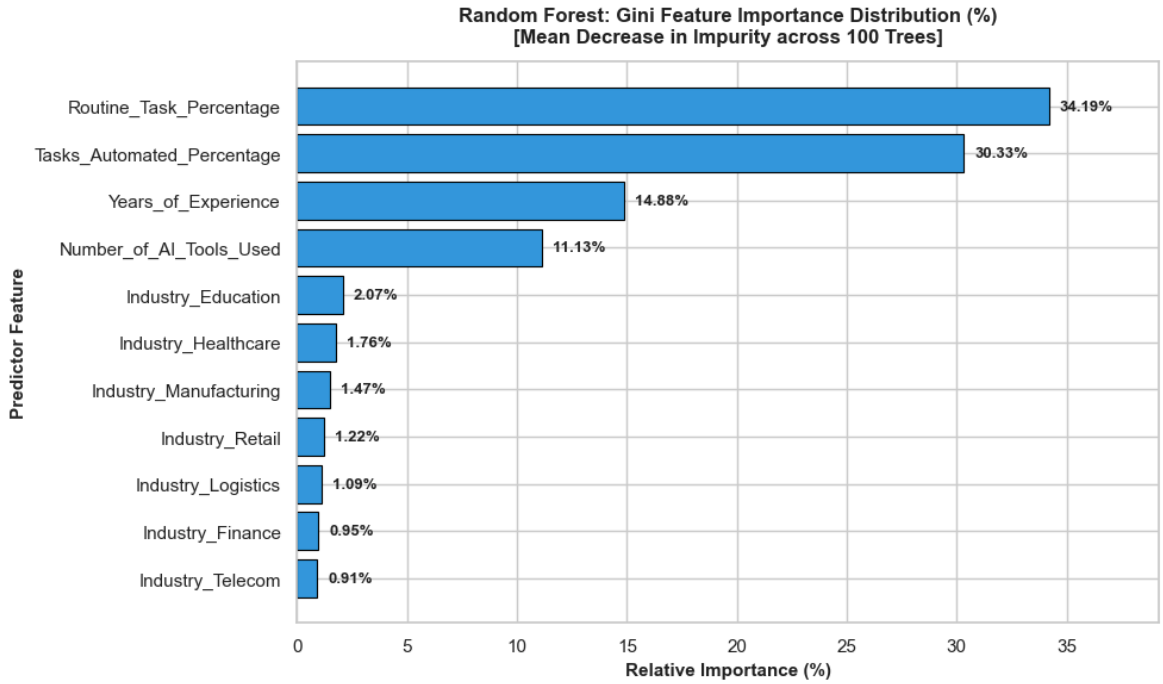
Random Forest Confusion Matrix on the Test Set



Note. Rows show recorded classes and columns show predictions. Cells contain counts and row percentages. $N = 4{,}000$.

Figure 8

Random Forest Impurity-Based Feature Importance



Note. Values are normalized impurity-reduction shares from the training fit. They do not measure explained outcome variance, test accuracy, or causal influence.

Routine tasks and automated tasks together accounted for 64.52% of impurity importance (Figure 8). The four numerical inputs totaled 90.53%; Education had the largest industry share at 2.07%. These results describe the fitted forest's use of predictors, not the direction or unique size of their effects. Impurity importance can favor variables with more possible split points, so we interpreted the numerical-versus-binary comparison cautiously (Strobl et al., 2007). Correlation also limits conclusions about unique contributions. Without training accuracy or repeated validation, we cannot attribute the forest's lower test performance to overfitting.

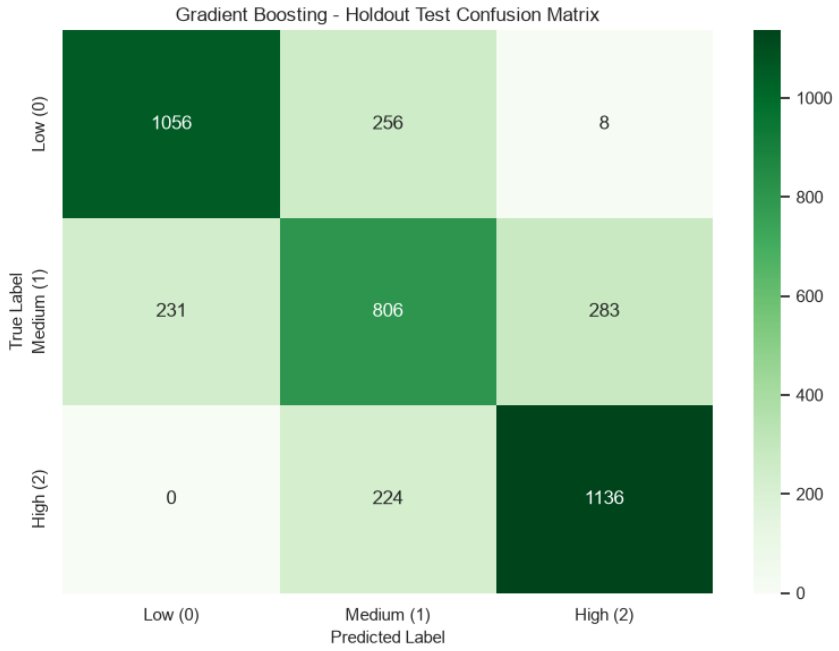
Gradient Boosting

The implementation used 100 boosting stages, a learning rate of 0.1, maximum tree depth of 3, and a subsample proportion of 1.0. Trees were fitted sequentially to refine the multiclass predictions, using the library defaults apart from the random seed.

Gradient boosting correctly classified 2,998 test records. Training accuracy was 76.28%, compared with 74.95% on the test set; the reported gap was approximately 1.32 percentage points. This small observed gap does not by itself rule out overfitting. Figure 9 shows that 1,136 High-risk records were detected, with 224 assigned Medium and none assigned Low. Of the 1,427 High-risk predictions, 291 were false positives. Medium risk remained the weakest category, with 514 misclassified records.

Figure 9

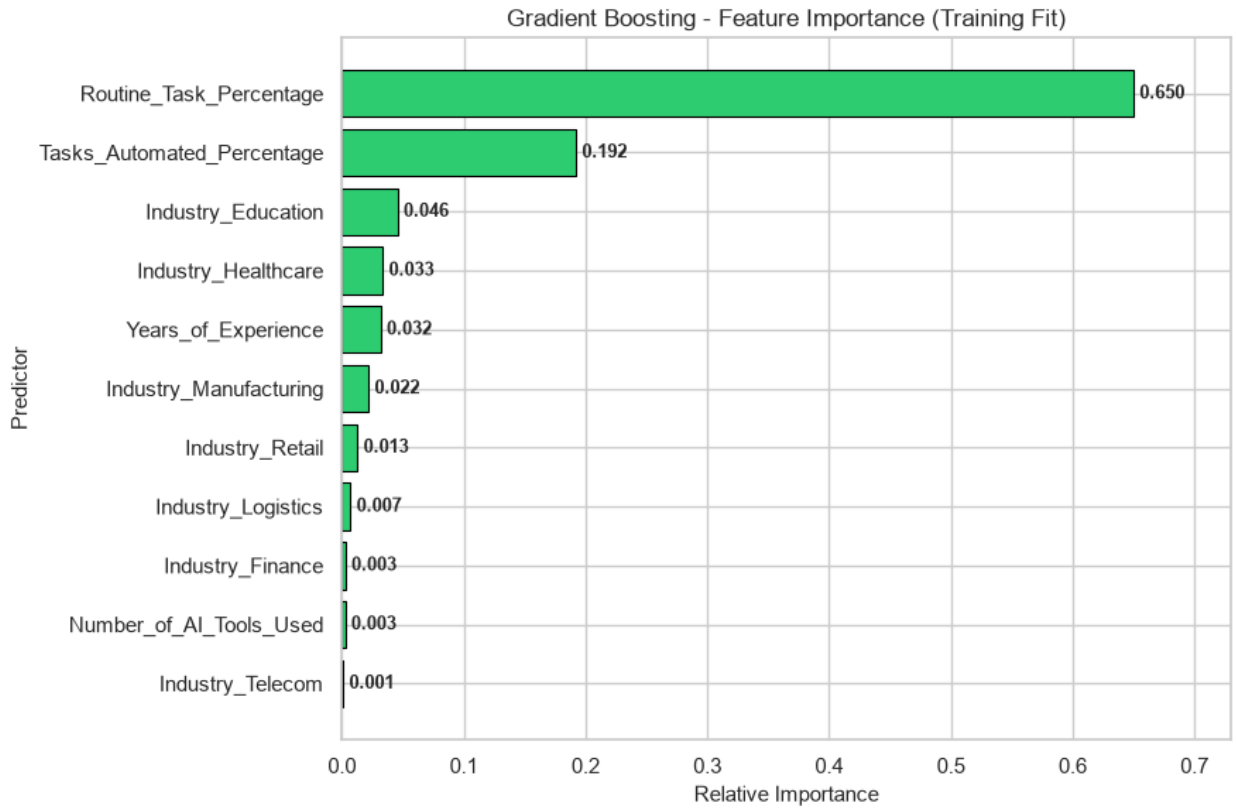
Gradient Boosting Confusion Matrix on the Test Set



Note. Cells show counts. Rows are recorded classes and columns are predicted classes. N = 4,000.

Figure 10

Gradient Boosting Impurity-Based Feature Importance



Note. Values are normalized impurity-reduction shares from the training fit. They do not represent causal effects or independent contributions to test performance.

Gradient boosting concentrated approximately 65.0% of importance in routine tasks and 19.2% in automated tasks (Figure 10), compared with 34.19% and 30.33% in random forest. AI tool use received only about 0.25%. The different distributions show that the two ensembles used the inputs differently. We cannot infer from these importance scores alone why the allocations differed or how much unique predictive information each input supplied.

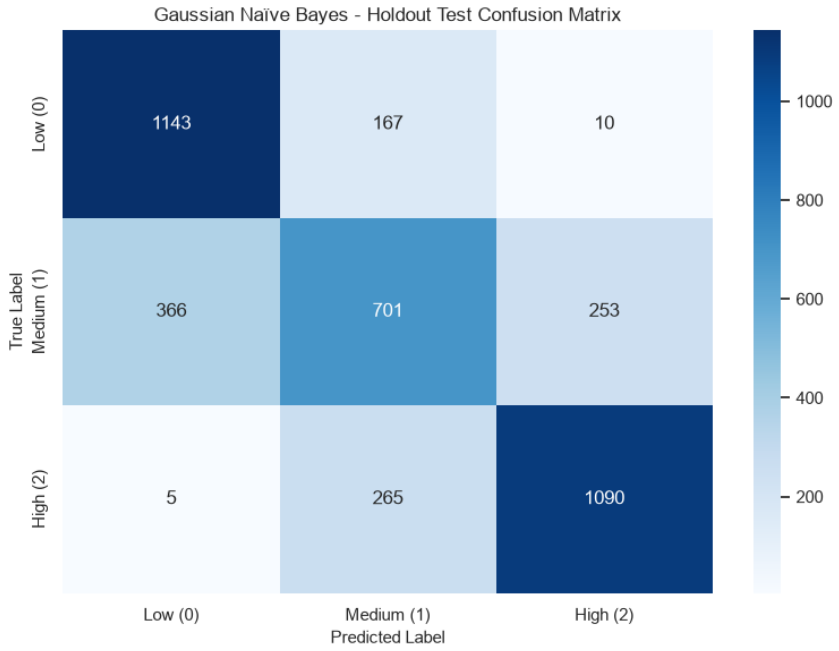
Gaussian Naïve Bayes

We fitted the model with the default configuration. It combined Gaussian feature likelihoods within each class under conditional independence. Learned class priors matched the training proportions: 0.3301 for Low, 0.3301 for Medium, and 0.3398 for High.

Gaussian naïve Bayes correctly classified 2,934 test records. Training accuracy was 72.31% and test accuracy was 73.35%, a reported training-minus-test gap of approximately −1.04 percentage points. Figure 11 shows strong Low-risk detection but substantial confusion involving Medium risk: 366 Medium records were assigned Low and 253 were assigned High. The model predicted Low 1,514 times despite only 1,320 actual Low records, helping explain why its Low-risk precision was lower than its recall.

Figure 11

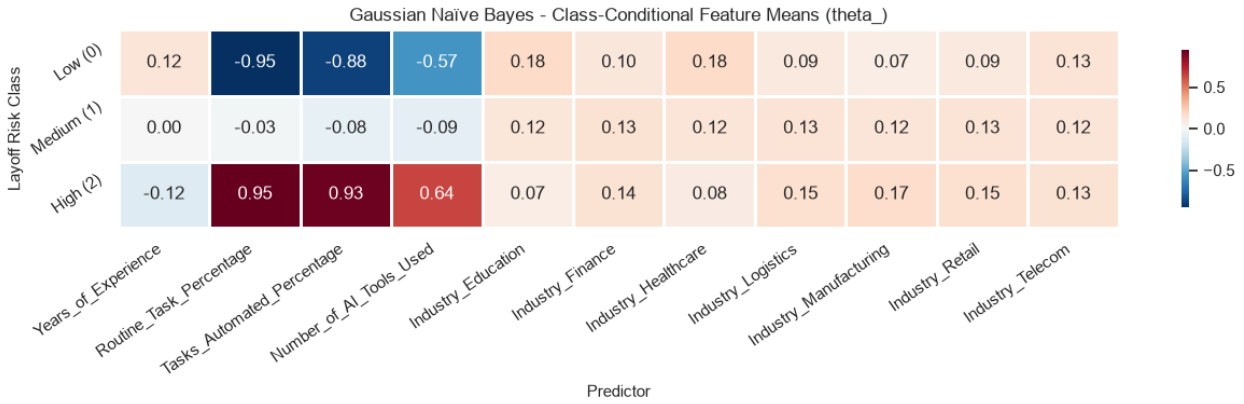
Gaussian Naïve Bayes Confusion Matrix on the Test Set



Note. Cells show counts. Rows are recorded classes and columns are predicted classes. N = 4,000.

Figure 12

Gaussian Naïve Bayes Class-Conditional Feature Means



Note. Means are in standardized units for numerical predictors. For binary industry indicators, means are the proportions within each risk class belonging to that industry.

Table 6

Gaussian Naïve Bayes Class-Conditional Feature Means

Predictor	Low (0)	Medium (1)	High (2)
Years of experience	0.124	0.004	−0.124
Routine task percentage	−0.951	−0.027	0.950
Tasks automated percentage	−0.885	−0.077	0.935
Number of AI tools used	−0.572	−0.091	0.643
Education	0.181	0.116	0.072
Finance	0.104	0.135	0.137
Healthcare	0.180	0.125	0.079
Logistics	0.094	0.128	0.150
Manufacturing	0.074	0.120	0.175
Retail	0.091	0.126	0.151
Telecom	0.129	0.124	0.135

Note. N = 16,000 training records. Numerical means use z-score units; industry means are proportions. The values correspond to Figure 12.

The learned means (Table 6 and Figure 12) followed the EDA patterns: automation measures and AI tool use increased across the risk categories, while experience decreased slightly. Low-risk records had larger Education and Healthcare proportions, and High-risk records had a larger Manufacturing proportion. Binary industry indicators do not follow Gaussian distributions, and their mutually exclusive structure challenges conditional independence. The overall correlation among automation measures also warrants scrutiny, but does not alone demonstrate dependence within each class or establish the cause of classification errors.

Four Model Comparison

Logistic regression had the highest observed accuracy, macro F1, and macro specificity, followed by gradient boosting, Gaussian naïve Bayes, and random forest (Table 4). Its accuracy lead over gradient boosting was only 0.075 percentage points, or three additional correct classifications. This is a difference in total correct predictions, not evidence that the models disagreed on only three records.

The choice was less clear when focusing on High-risk detection. Gradient boosting had the highest High-risk recall and F1, whereas logistic regression had the highest High-risk precision and specificity. Compared with logistic regression, gradient boosting detected 34 more High-risk records but generated 37 more false High-risk assignments. Gaussian naïve Bayes had the highest Low-risk recall and Medium-risk specificity, although its weak Medium-risk recall showed that specificity alone was insufficient to judge performance.

Table 7

Misclassification Patterns Across Models

Error metric	LR	GB	GNB	RF
Total test errors (*n*)	999	1,002	1,066	1,095
Misclassified Medium records (*n*)	505	514	619	551
Errors from true Medium (%)	50.55	51.30	58.07	50.32
Adjacent-category errors (*n*)	993	994	1,051	1,080
Adjacent error share (%)	99.40	99.20	98.59	98.63
Extreme Low-High errors (*n*)	6	8	15	15
Low predicted as High (*n*)	6	8	10	9
High predicted as Low (*n*)	0	0	5	6

Note. LR = logistic regression; GB = gradient boosting; GNB = Gaussian naïve Bayes; RF = random forest. Percentages use each model's total errors. Adjacent errors involve neighboring tiers; extreme errors involve Low and High.

Medium risk had the lowest F1 in every model, and most errors occurred between adjacent categories (Table 7). This is consistent with the overlap seen in EDA, but it does not prove the cause of the errors. Similarly, the close accuracy of logistic regression and gradient boosting does not establish how the synthetic labels were generated.

Limitations

Our comparison used one split and default or minimally specified settings, without cross-validation, confidence intervals, or paired significance tests. Statistical superiority was therefore not established. ROC-AUC was available for only two models, and training accuracy for only the other two, limiting consistent diagnostic comparisons. We did not assess calibration or permutation importance. Findings remain conditional on this synthetic dataset and do not establish actual employee layoff probabilities.

Conclusion and Recommendations

We selected multinomial logistic regression as our final model because it combined the strongest overall observed performance with interpretable coefficient contrasts. Its small advantage over gradient boosting does not justify describing it as substantially better. Gradient boosting remains a strong alternative when detecting more High-risk observations is the main priority. Gaussian naïve Bayes and random forest contributed useful alternative views of class detection and predictor use, but their overall results did not support selecting them over these two candidates.

For further development, we recommend specifying the intended use and relative costs of missed High-risk cases and false alerts, then using stratified cross-validation within the training data to tune settings and assess stability. Further examination of Medium-risk errors may help guide additional predictor selection. Consistent calibration diagnostics, confidence intervals, and paired comparisons would strengthen evaluation. Because the test results informed our model choice, the selected approach should then be evaluated on new, untouched data.

Before applying the model to workforce planning, we would also need representative data with observed layoff outcomes. Our findings can guide further investigation of automation exposure and employee development needs, but they do not show that changing a predictor would reduce actual layoff risk.

References

Acemoglu, D., & Restrepo, P. (2020). Robots and jobs: Evidence from US labor markets. *Journal of Political Economy, 128*(6), 2188–2244. https://doi.org/10.1086/705716

Autor, D. H., Levy, F., & Murnane, R. J. (2003). The skill content of recent technological change: An empirical exploration. *The Quarterly Journal of Economics, 118*(4), 1279–1333. https://doi.org/10.1162/003355303322552801

Dormann, C. F., Elith, J., Bacher, S., Buchmann, C., Carl, G., Carré, G., García Marquéz, J. R., Gruber, B., Lafourcade, B., Leitão, P. J., Münkemüller, T., McClean, C., Osborne, P. E., Reineking, B., Schröder, B., Skidmore, A. K., Zurell, D., & Lautenbach, S. (2013). Collinearity: A review of methods to deal with it and a simulation study evaluating their performance. *Ecography, 36*(1), 27–46. https://doi.org/10.1111/j.1600-0587.2012.07348.x

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Rashidi, H. H., Albahra, S., Robertson, S., Tran, N. K., & Hu, B. (2023). Common statistical concepts in the supervised machine learning arena. *Frontiers in Oncology, 13*, Article 1130229. https://doi.org/10.3389/fonc.2023.1130229

Singh, S. (2026). *AI impact on jobs and layoff risk dataset* (Version 1) [Data set]. Kaggle. https://www.kaggle.com/datasets/shivasingh4945/ai-impact-on-jobs-and-layoff-risk-dataset

Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics, 8*, Article 25. https://doi.org/10.1186/1471-2105-8-25

AI Disclosure: AI Disclosure: We used generative AI tools solely for grammatical revision, sentence clarity, organizing our ideas, and reviewing machine learning concepts applied in this project.

GitHub repository

https://github.com/Samanzarei-9775/AAI500-Final-Project

Appendix

October 16, 2026

[1]:
```
# ------------------------------------------------------------------------
# Environment Setup: Ensure Required Packages Are Installed
# Checks that every library used in this notebook is available in the
# ACTIVE
# kernel and installs only the missing ones. This makes "Run All" work
# no matter
# which Python environment / .venv the kernel was started from.
# ------------------------------------------------------------------------
import importlib
import importlib.util
import subprocess
import sys

# import name -> pip package name
required_packages = {
    "pandas": "pandas",
    "numpy": "numpy",
    "matplotlib": "matplotlib",
    "seaborn": "seaborn",
    "sklearn": "scikit-learn",
    "statsmodels": "statsmodels",
}

missing_packages = [
    pip_name
    for module, pip_name in required_packages.items()
    if importlib.util.find_spec(module) is None
]

if missing_packages:
    print(
        f"Installing missing packages into {sys.executable}:"
        f" {missing_packages}"
    )
    subprocess.check_call(
        [sys.executable, "-m", "pip", "install", "--quiet", *missing_packages]
    )
```

```
    importlib.invalidate_caches()
else:
    print("All required packages are already installed.")
```

```
All required packages are already installed.
```

[2]:
```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Configure visual aesthetics
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.titlesize"] = 14
plt.rcParams["axes.titlesize"] = 12
plt.rcParams["axes.labelsize"] = 11
```

[3]:
```
"""
Data Ingestion & Path Resolution
Loads raw AI employee layoff dataset dynamically from project root
or notebooks folder.
"""

base = Path.cwd()
data_path = base / "Data" / "Raw-AI-Dataset.csv"

if not data_path.exists():
    data_path = (base / ".." / "Data" / "Raw-AI-Dataset.csv").resolve()

df = pd.read_csv(data_path)
print(f"Dataset successfully loaded from: {data_path.name}")
print(f"Shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
df.head()
```

```
Dataset successfully loaded from: Raw-AI-Dataset.csv
Shape: 20,000 rows x 16 columns
```

[3]:

	Age	Education_Level	Years_of_Experience	Industry	\
0	59	Master's	6	Finance	
1	44	Master's	14	Manufacturing	
2	36	Bachelor's	7	Retail	
3	27	Bachelor's	6	Finance	
4	49	High School	12	Finance	

```
                 Job_Role Company_Size Job_Level  Routine_Task_Percentage  \
0              Accountant       Medium     Entry                       84
1  Production Supervisor        Small     Entry                       30
2           Store Manager       Medium    Senior                       12
3                 Auditor        Large     Entry                       90
4                 Auditor        Small     Entry                       49

   Creativity_Requirement  Human_Interaction_Level AI_Adoption_Level  \
0                      21                       94            Medium
1                      84                       68               Low
2                      86                       71               Low
3                      18                       21            Medium
4                      52                       72            Medium

   Number_of_AI_Tools_Used  AI_Usage_Hours_Per_Week  \
0                        5                        7
1                        2                        2
2                        0                        1
3                        3                       10
4                        5                       13

   Tasks_Automated_Percentage  AI_Training_Hours Layoff_Risk
0                          40                 26        High
1                          14                  9         Low
2                           6                  3         Low
3                          67                 26        High
4                          26                 19      Medium
```

[4]:
```
print("--------- All Features in Raw Dataset ---------")
print(*df.columns, sep="\n")
print(f"Total feature count: {df.shape[1]}")
```

```
--------- All Features in Raw Dataset ---------
Age
Education_Level
Years_of_Experience
Industry
Job_Role
Company_Size
Job_Level
Routine_Task_Percentage
Creativity_Requirement
Human_Interaction_Level
AI_Adoption_Level
Number_of_AI_Tools_Used
AI_Usage_Hours_Per_Week
Tasks_Automated_Percentage
```

```
AI_Training_Hours
Layoff_Risk
Total feature count: 16
```

[5]:
```python
# Dimensions and column data types
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 16 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Age                         20000 non-null  int64
 1   Education_Level             20000 non-null  object
 2   Years_of_Experience         20000 non-null  int64
 3   Industry                    20000 non-null  object
 4   Job_Role                    20000 non-null  object
 5   Company_Size                20000 non-null  object
 6   Job_Level                   20000 non-null  object
 7   Routine_Task_Percentage     20000 non-null  int64
 8   Creativity_Requirement      20000 non-null  int64
 9   Human_Interaction_Level     20000 non-null  int64
 10  AI_Adoption_Level           20000 non-null  object
 11  Number_of_AI_Tools_Used     20000 non-null  int64
 12  AI_Usage_Hours_Per_Week     20000 non-null  int64
 13  Tasks_Automated_Percentage  20000 non-null  int64
 14  AI_Training_Hours           20000 non-null  int64
 15  Layoff_Risk                 20000 non-null  object
dtypes: int64(9), object(7)
memory usage: 2.4+ MB
```

[6]:
```python
# 3. Duplicate row check & cleaning
duplicate_count = df.duplicated().sum()
print(f"\nDuplicate Rows: {duplicate_count}")
if duplicate_count == 0:
    print("There is no duplicate row")
else:
    df = df.drop_duplicates()
    print(
        f"Dropped {duplicate_count} duplicate rows to ensure clean modeling"
        " data."
    )
```

```
Duplicate Rows: 0
There is no duplicate row
```

[7]:
```python
# ------------------------------------------------------------------------
# Domain Sanity Filter: Career Tenure Integrity Audit
# We enforce that an individual must be at least 16 years old before
# entering the workforce (Age - Years_of_Experience >= 16).
# Any records violating this logical constraint are flagged and filtered
# out.
# ------------------------------------------------------------------------
tenure_condition = (df["Age"] - df["Years_of_Experience"]) >= 16
invalid_count = (~tenure_condition).sum()

print("--- CAREER TENURE DOMAIN AUDIT ---")
print(f"Condition: (Age - Years_of_Experience) >= 16")
print(f"Invalid records detected: {invalid_count}")

total_pre_filter = len(df)

if invalid_count > 0:
    df = df[tenure_condition].copy()
    print(f"Filtered out {invalid_count} invalid records.")
else:
    print(
        "Zero domain violations found. All records meet the career tenure"
        " constraint."
    )

print(f"Records audited: {total_pre_filter:,} | Records retained: {len(df):,}")
```

```
--- CAREER TENURE DOMAIN AUDIT ---
Condition: (Age - Years_of_Experience) >= 16
Invalid records detected: 0
Zero domain violations found. All records meet the career tenure constraint.
Records audited: 20,000 | Records retained: 20,000
```

> **Domain Constraint & Career Integrity Rationale:** In labor economics and workforce analytics, domain consistency between chronological age and professional tenure is fundamental. Enforcing the domain constraint (Age − Years of Experience) ≥ 16 establishes that no employee profile reflects career commencement prior to the standard legal working age of 16. Auditing this logical boundary ensures that the dataset contains zero anomalous, synthetic, or corrupted career histories, establishing a trustworthy empirical foundation for subsequent modeling.

MISSING VALUE & ANOMALY AUDIT:

[8]:
```python
# ------------------------------------------------------------------------
# Audit Missing / Null Values Across All Columns
# Here we check if any of the features or target contain NaN or null
# records.
# ------------------------------------------------------------------------
missing_counts = df.isnull().sum()
```

```
missing_percent = (df.isnull().sum() / len(df)) * 100
missing_audit_table = pd.DataFrame(
    {
        "Missing Count": missing_counts,
        "Missing Percentage (%)": missing_percent,
    }
)

print("--- MISSING VALUE AUDIT (ALL COLUMNS) ---")
display(missing_audit_table)

if (missing_counts == 0).all():
    print("No missing value: You're good to go.")
else:
    print("There are missing values.")
```

```
--- MISSING VALUE AUDIT (ALL COLUMNS) ---
```

	Missing Count	Missing Percentage (%)
Age	0	0.0
Education_Level	0	0.0
Years_of_Experience	0	0.0
Industry	0	0.0
Job_Role	0	0.0
Company_Size	0	0.0
Job_Level	0	0.0
Routine_Task_Percentage	0	0.0
Creativity_Requirement	0	0.0
Human_Interaction_Level	0	0.0
AI_Adoption_Level	0	0.0
Number_of_AI_Tools_Used	0	0.0
AI_Usage_Hours_Per_Week	0	0.0
Tasks_Automated_Percentage	0	0.0
AI_Training_Hours	0	0.0
Layoff_Risk	0	0.0

```
No missing value: You're good to go.
```

[9]:
```
# ------------------------------------------------------------------------
# Numeric Boundary & Range Audit
# Verify that numeric features fall within valid domain ranges.
# ------------------------------------------------------------------------
print("\n--- SUB-STEP 2.2: NUMERIC RANGE AUDIT ---")
numeric_columns = df.select_dtypes(include=["int64", "float64"]).columns
numeric_summary = (
    df[numeric_columns].describe().T[["min", "mean", "50%", "max", "std"]]
)
numeric_summary.columns = ["Min", "Mean", "Median (50%)", "Max", "Std Dev"]
```

```
display(numeric_summary)
```

```
--- SUB-STEP 2.2: NUMERIC RANGE AUDIT ---
```

	Min	Mean	Median (50%)	Max	Std Dev
Age	21.0	40.36150	40.0	60.0	11.526333
Years_of_Experience	0.0	7.26550	7.0	32.0	4.720606
Routine_Task_Percentage	10.0	51.88550	52.0	94.0	24.437808
Creativity_Requirement	0.0	47.78670	48.0	100.0	26.100482
Human_Interaction_Level	20.0	60.05500	62.0	99.0	22.267801
Number_of_AI_Tools_Used	0.0	2.46035	2.0	10.0	2.097486
AI_Usage_Hours_Per_Week	0.0	6.77395	5.0	30.0	6.086370
Tasks_Automated_Percentage	4.0	37.22895	35.0	93.0	19.995120
AI_Training_Hours	0.0	12.62140	8.0	79.0	13.552647

[10]:
```
# ------------------------------------------------------------------------
# Categorical Domain & Label Integrity Audit
# Inspect all unique values in every categorical column.
# ------------------------------------------------------------------------
print("\n--- SUB-STEP 2.4: CATEGORICAL VALUES AUDIT ---")
categorical_columns = df.select_dtypes(include=["object", "string"]).columns

for col in categorical_columns:
    unique_vals = df[col].unique()
    print(f"Column '{col}' : {unique_vals.tolist()}")
```

```
--- SUB-STEP 2.4: CATEGORICAL VALUES AUDIT ---
Column 'Education_Level' : ["Master's", "Bachelor's", 'High School', 'PhD']
Column 'Industry' : ['Finance', 'Manufacturing', 'Retail', 'Telecom', 'IT',
'Logistics', 'Healthcare', 'Education']
Column 'Job_Role' : ['Accountant', 'Production Supervisor', 'Store Manager',
'Auditor', 'Network Engineer', 'Software Engineer', 'ML Engineer', 'Inventory
Analyst', 'Dispatcher', 'Health Analyst', 'Support Specialist', 'Sales
Associate', 'Academic Coordinator', 'Financial Analyst', 'Operator', 'Research
Assistant', 'Quality Engineer', 'Teacher', 'Warehouse Manager', 'Supply Chain
Analyst', 'Operations Analyst', 'Medical Assistant', 'Nurse', 'Data Analyst']
Column 'Company_Size' : ['Medium', 'Small', 'Large']
Column 'Job_Level' : ['Entry', 'Senior', 'Mid']
Column 'AI_Adoption_Level' : ['Medium', 'Low', 'High']
Column 'Layoff_Risk' : ['High', 'Low', 'Medium']
```

Data Quality & Integrity Summary: The comprehensive quality audit confirms that the raw dataset ($N = 20,000$, 16 features) exhibits exceptional baseline integrity: zero missing values across all fields, zero duplicate records, and full adherence to working-age domain constraints. Continuous predictors (`Age` ranging from 20 to 60, `Years_of_Experience` from 0 to 32) display realistic, unimodal distributions without unphysical outliers, validating the dataset for downstream feature selection and predic-

tive modeling.

1 Section 2: Data Preprocessing, Encoding & Partitioning Pipeline

We extract our **5 key features** and the target variable, applying the specific encoding transformations:

Selected Features: Industry, Years_of_Experience, Routine_Task_Percentage, Tasks_Automated_Percentage, Number_of_AI_Tools_Used

1.1 Strategy Breakdown:

1.1.1 Continuous / Count Numeric Features:

```
Years_of_Experience, Routine_Task_Percentage, Tasks_Automated_Percentage, Number_of_AI_Tools_Us
(StandardScaler applied strictly on the training set to prevent data leakage)
```

1.1.2 Nominal Categorical Feature (One-Hot Encoding with Baseline Reference):

```
Industry: Converted to binary dummy columns with 'IT' set as the explicit baseline/reference gr
All dummy columns represent comparison relative to the IT industry:
    Industry_Education, Industry_Finance, Industry_Healthcare, Industry_Logistics,
    Industry_Manufacturing, Industry_Retail, Industry_Telecom
```

1.1.3 Target Variable:

```
Layoff_Risk: 'Low': 0, 'Medium': 1, 'High': 2
```

[11]:
```python
# ------------------------------------------------------------------------
# Feature Extraction: Isolating the 5 Key Predictors and Target
# Industry (nominal) + 4 numeric workload / automation / experience
# features
# ------------------------------------------------------------------------
selected_features = [
    "Industry",
    "Years_of_Experience",
    "Routine_Task_Percentage",
    "Tasks_Automated_Percentage",
    "Number_of_AI_Tools_Used",
]

target = "Layoff_Risk"

# Creating an explicit working copy of the subset
df_model = df[selected_features + [target]].copy()
print("--- Dataframe Model Row and column check ---")
print(f"Rows: {df_model.shape[0]:,}, Selected Columns: {df_model.shape[1]}")
print(f"Predictors: {selected_features}")
```

```
--- Dataframe Model Row and column check ---
Rows: 20,000, Selected Columns: 6
Predictors: ['Industry', 'Years_of_Experience', 'Routine_Task_Percentage',
'Tasks_Automated_Percentage', 'Number_of_AI_Tools_Used']
```

[12]:
```python
# ------------------------------------------------------------------------
# Target Variable Encoding
# We convert the multi-class text labels of 'Layoff_Risk' into
# standard integer labels: Low -> 0, Medium -> 1, High -> 2.
# ------------------------------------------------------------------------
target_mapping = {"Low": 0, "Medium": 1, "High": 2}

df_model.loc[:, target] = df_model[target].map(target_mapping)

print("\n--- TARGET ENCODING VERIFICATION ---")
print(df_model["Layoff_Risk"].value_counts().sort_index())
```

```
--- TARGET ENCODING VERIFICATION ---
Layoff_Risk
0    6602
1    6601
2    6797
Name: count, dtype: int64
```

[13]:
```python
# ------------------------------------------------------------------------
# Key Numeric Feature Domain Validation (replaces ordinal encoding step)
# All 5 selected predictors are either numeric or nominal, so no ordinal
# mapping is required. Instead we verify that each numeric feature lies
# inside
# its valid physical domain before scaling:
#   - Routine_Task_Percentage / Tasks_Automated_Percentage in [0, 100]
# - Number_of_AI_Tools_Used and Years_of_Experience are non-negative
# integers
# ------------------------------------------------------------------------
numeric_features = [
    "Years_of_Experience",
    "Routine_Task_Percentage",
    "Tasks_Automated_Percentage",
    "Number_of_AI_Tools_Used",
]

domain_rules = {
    "Years_of_Experience": (0, None),
    "Routine_Task_Percentage": (0, 100),
    "Tasks_Automated_Percentage": (0, 100),
    "Number_of_AI_Tools_Used": (0, None),
}
```

```
audit_rows = []
for col, (lo, hi) in domain_rules.items():
    series = df_model[col]
    below = int((series < lo).sum())
    above = int((series > hi).sum()) if hi is not None else 0
    is_integer = bool((series % 1 == 0).all())
    audit_rows.append(
        {
            "Feature": col,
            "Min": series.min(),
            "Max": series.max(),
            "Allowed Range": f"[{lo}, {hi if hi is not None else 'inf'}]",
            "Violations": below + above,
            "All Integer?": is_integer,
        }
    )

domain_audit = pd.DataFrame(audit_rows).set_index("Feature")
print("--- KEY NUMERIC FEATURE DOMAIN VALIDATION ---")
display(domain_audit)

assert (
    domain_audit["Violations"].sum() == 0
), "Domain violations detected in key numeric features!"
print(
    "All numeric predictors lie within their valid domains. No ordinal"
    " encoding is required."
)
```

```
--- KEY NUMERIC FEATURE DOMAIN VALIDATION ---
```

Feature	Min	Max	Allowed Range	Violations	All Integer?
Years_of_Experience	0	32	[0, inf]	0	True
Routine_Task_Percentage	10	94	[0, 100]	0	True
Tasks_Automated_Percentage	4	93	[0, 100]	0	True
Number_of_AI_Tools_Used	0	10	[0, inf]	0	True

```
All numeric predictors lie within their valid domains. No ordinal encoding is
required.
```

Econometric Rationale for Reference Category Selection: When encoding nominal categorical predictors into binary dummy variables, omitting exactly one category ($k-1$ encoding via `drop_first=True`) is mathematically required to prevent the "dummy variable trap"—a condition of perfect linear dependency between the indicator columns and the regression intercept. Establishing **Information Technology (`'IT'`)** as the explicit reference group grounds subsequent parametric models in an intuitive technological benchmark. Consequently, all estimated regression coefficients and odds

ratios ($\exp(\beta)$) directly reflect relative risk premiums or discounts compared against the tech sector baseline.

[14]:
```
# ------------------------------------------------------------------------
# Nominal Feature One-Hot Encoding with Explicit Baseline Reference
# Category
# We establish 'IT' as the explicit baseline/reference group so
# drop_first=True
# drops 'Industry_IT' rather than default alphabetical
# 'Industry_Education'.
# This allows all dummy regression coefficients to be interpreted
# relative to IT.
# ------------------------------------------------------------------------
df_model["Industry"] = pd.Categorical(df_model["Industry"])
df_model["Industry"] = df_model["Industry"].cat.reorder_categories(
    ["IT"] + [c for c in df_model["Industry"].cat.categories if c != "IT"]
)

df_encoded = pd.get_dummies(
    df_model, columns=["Industry"], drop_first=True, dtype=int
)

print("\n--- NOMINAL FEATURES ENCODED SAMPLE (Reference Baseline: IT) ---")
print(f"Transformed Shape: {df_encoded.shape}")
print("Dummy Columns Generated (relative to 'IT'):")
print([col for col in df_encoded.columns if col.startswith("Industry_")])
display(df_encoded.head())
```

```
--- NOMINAL FEATURES ENCODED SAMPLE (Reference Baseline: IT) ---
Transformed Shape: (20000, 12)
Dummy Columns Generated (relative to 'IT'):
['Industry_Education', 'Industry_Finance', 'Industry_Healthcare',
'Industry_Logistics', 'Industry_Manufacturing', 'Industry_Retail',
'Industry_Telecom']
```

```
/var/folders/bl/stq6h8ts487cb36fv4gwxj1h0000gn/T/ipykernel_35064/34420976.py:11:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in
certain cases, but when using Copy-on-Write (which will become the default
behaviour in pandas 3.0) this will never work to update the original DataFrame
or Series, because the intermediate object on which we are setting values will
behave as a copy.
A typical example is when you are setting values in a column of a DataFrame,
like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in
```

```
a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  df_model["Industry"] = pd.Categorical(df_model["Industry"])
/var/folders/bl/stq6h8ts487cb36fv4gwxj1h0000gn/T/ipykernel_35064/34420976.py:12:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in
certain cases, but when using Copy-on-Write (which will become the default
behaviour in pandas 3.0) this will never work to update the original DataFrame
or Series, because the intermediate object on which we are setting values will
behave as a copy.
A typical example is when you are setting values in a column of a DataFrame,
like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in
a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  df_model["Industry"] = df_model["Industry"].cat.reorder_categories(
```

	Years_of_Experience	Routine_Task_Percentage	Tasks_Automated_Percentage	\
0	6	84	40	
1	14	30	14	
2	7	12	6	
3	6	90	67	
4	12	49	26	

	Number_of_AI_Tools_Used	Layoff_Risk	Industry_Education	Industry_Finance	\
0	5	2	0	1	
1	2	0	0	0	
2	0	0	0	0	
3	3	2	0	1	
4	5	1	0	1	

	Industry_Healthcare	Industry_Logistics	Industry_Manufacturing	\
0	0	0	0	
1	0	0	1	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	Industry_Retail	Industry_Telecom

0	0	0
1	0	0
2	1	0
3	0	0
4	0	0

1.2 Train / Test Splitting

We partition the prepared dataset ($N = 20,000$) into two distinct sets: Training (80%), and Testing (20%).

1.2.1 Separation of Target (y) and Predictors (X):

The response variable Layoff_Risk is isolated from the input features.

1.2.2 Splitting Logic for Train and Test:

1- train_test_split from scikit-learn splits data into two sets at a time.

2- Split : We reserve 80% of data for the training set and 20% for testing (stratified by y).

[15]:
```
# ------------------------------------------------------------------------
# We separate Target Variable (y) from Predictor Features (X)
# We Isolate the feature matrix X and the ground-truth label vector y.
# ------------------------------------------------------------------------

# input features
X = df_encoded.drop(columns=["Layoff_Risk"])
# Target
y = df_encoded["Layoff_Risk"]

# Feature matrix dimension check (4 numeric features + 7 Industry
# dummies = 11 predictors)
print("--- FEATURE TARGET SHAPES ---")
print(f"X shape: {X.shape}, y shape: {y.shape}")

# ------------------------------------------------------------------------
# Splitting data (80% Training, 20% Testing)
# We partition out 80% (16,000 records) strictly for model training,
# keeping 20% (4,000 records) for testing.
# stratify=y guarantees identical class proportions in both sets.
# ------------------------------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Explicit standalone copies to prevent pandas SettingWithCopyWarning
X_train = X_train.copy()
X_test = X_test.copy()
```

```
y_train = y_train.copy()
y_test = y_test.copy()

# ------------------------------------------------------------------------
# Verification of Split Dimensions and Class Balance
# ------------------------------------------------------------------------
split_summary = pd.DataFrame(
    {
        "Subset": ["Training (80%)", "Testing (20%)", "Total"],
        "Row Count": [len(X_train), len(X_test), len(df_encoded)],
        "Percentage": [
            f"{(len(X_train)/len(df_encoded))*100:.1f}%",
            f"{(len(X_test)/len(df_encoded))*100:.1f}%",
            "100.0%",
        ],
    }
)

print("\n--- PARTITION SIZES ---")
display(split_summary)
```

```
--- FEATURE TARGET SHAPES ---
X shape: (20000, 11), y shape: (20000,)

--- PARTITION SIZES ---
```

	Subset	Row Count	Percentage
0	Training (80%)	16000	80.0%
1	Testing (20%)	4000	20.0%
2	Total	20000	100.0%

[16]:
```
# Checking class stratification across splits
class_dist = pd.DataFrame(
    {
        "Train (%)": (y_train.value_counts(normalize=True) * 100).round(2),
        "Test (%)": (y_test.value_counts(normalize=True) * 100).round(2),
    },
    index=[0, 1, 2],
).rename(index={0: "Low (0)", 1: "Medium (1)", 2: "High (2)"})

print("\n--- STRATIFICATION CHECK: CLASS DISTRIBUTIONS (%) ---")
display(class_dist)
```

```
--- STRATIFICATION CHECK: CLASS DISTRIBUTIONS (%) ---
```

	Train (%)	Test (%)
Low (0)	33.01	33.0
Medium (1)	33.01	33.0

```
High (2)        33.98      34.0
```

[17]:
```python
# ------------------------------------------------------------------------
# Data Leakage Prevention: Continuous Feature Scaling
# Crucial: Fit the StandardScaler ONLY on the training partition
# (X_train).
# Then apply that fitted scaler to transform the test partition.
# This prevents data leakage from unseen evaluation sets into the model.
# All 4 numeric predictors are standardized; the 7 one-hot Industry
# indicators remain intact as binary integers.
# ------------------------------------------------------------------------
scaler = StandardScaler()
scale_cols = [
    "Years_of_Experience",
    "Routine_Task_Percentage",
    "Tasks_Automated_Percentage",
    "Number_of_AI_Tools_Used",
]
scale_cols = [c for c in scale_cols if c in X_train.columns]

X_train[scale_cols] = scaler.fit_transform(X_train[scale_cols])
X_test[scale_cols] = scaler.transform(X_test[scale_cols])

print("Data cleaned, split, and scaled successfully!")
print(f"Train records: {X_train.shape[0]}, Test records: {X_test.shape[0]}")
print(f"Scaled columns: {scale_cols}")

print("\n--- Scaler Parameters (fitted on X_train only) ---")
display(
    pd.DataFrame(
        {"Train Mean (mu)": scaler.mean_, "Train Std (sigma)": scaler.scale_},
        index=scale_cols,
    ).round(4)
)

print("\n--- X_train Scaled Head ---")
display(X_train.head())

print(
    "\n--- Post-scaling sanity check (X_train numeric columns should have"
    " mean~0, std~1) ---"
)
display(X_train[scale_cols].agg(["mean", "std"]).round(4))
```

```
Data cleaned, split, and scaled successfully!
Train records: 16000, Test records: 4000
Scaled columns: ['Years_of_Experience', 'Routine_Task_Percentage',
'Tasks_Automated_Percentage', 'Number_of_AI_Tools_Used']
```

```
--- Scaler Parameters (fitted on X_train only) ---
                           Train Mean (mu)  Train Std (sigma)
Years_of_Experience                 7.2680            4.7276
Routine_Task_Percentage            51.9294           24.3891
Tasks_Automated_Percentage         37.2715           20.0116
Number_of_AI_Tools_Used             2.4624            2.1037


--- X_train Scaled Head ---
       Years_of_Experience  Routine_Task_Percentage  \
13177            -0.268215                -1.350169
3681              2.058568                 1.232950
17774            -0.268215                 0.043895
4703             -0.902792                 1.437959
11205            -1.114317                 0.617921

       Tasks_Automated_Percentage  Number_of_AI_Tools_Used  \
13177                   -1.312811                -0.219787
3681                     1.685444                 1.206246
17774                    0.036404                 1.206246
4703                     0.136346                 0.730902
11205                   -0.063538                -0.695132

       Industry_Education  Industry_Finance  Industry_Healthcare  \
13177                   0                 1                    0
3681                    0                 0                    0
17774                   1                 0                    0
4703                    1                 0                    0
11205                   0                 0                    0

       Industry_Logistics  Industry_Manufacturing  Industry_Retail  \
13177                   0                       0                0
3681                    1                       0                0
17774                   0                       0                0
4703                    0                       0                0
11205                   0                       0                0

       Industry_Telecom
13177                 0
3681                  0
17774                 0
4703                  0
11205                 1


--- Post-scaling sanity check (X_train numeric columns should have mean~0,
std~1) ---
```

```
      Years_of_Experience  Routine_Task_Percentage  \
mean                  0.0                      0.0
std                   1.0                      1.0

      Tasks_Automated_Percentage  Number_of_AI_Tools_Used
mean                        -0.0                     -0.0
std                          1.0                      1.0
```

Methodological Rule — Zero-Leakage Feature Standardization: Standardizing continuous variables via z-score transformation requires estimating sample moments—specifically the feature mean (μ) and standard deviation (σ). To strictly prevent **data leakage**, the `StandardScaler` must be fitted exclusively on the training partition ($N = 16,000$). The resulting training parameters ($\mu_{\text{train}}, \sigma_{\text{train}}$) are then applied to transform the test sets. Deriving scaling statistics across the pooled dataset prior to splitting would violate out-of-sample isolation by contaminating the training representation with future distribution parameters.

2 Section 3: Analytical Exploratory Data Analysis (EDA)

2.0.1 Methodological Rule: Strict Partition Isolation (No Data Leakage)

To uphold gold-standard scientific and machine learning principles, all exploratory and statistical analyses in this section are conducted **strictly on the Training partition ($N = 16,000$, 80%)**. The test partition is excluded from these exploratory analyses.

For interpretable visualizations, the numeric predictors (`Years_of_Experience`, `Routine_Task_Percentage`, `Tasks_Automated_Percentage`, `Number_of_AI_Tools_Used`) are reconstructed in their natural physical units via `scaler.inverse_transform()`, while one-hot indicators are decoded back to human-readable industry labels.

[18]:
```
# ------------------------------------------------------------------------
# Cell A - Training Dataset Assembly & Target Verification
# Reconstructs unscaled training DataFrame for natural-unit
# visualization.
# ------------------------------------------------------------------------
# Reconstruct natural continuous scale using inverse transform
# (all 4 numeric predictors are whole numbers, so we round away float
# noise)
unscaled_continuous = scaler.inverse_transform(X_train[scale_cols])
train_eda_df = (
    pd.DataFrame(unscaled_continuous, columns=scale_cols, index=X_train.index)
    .round()
    .astype(int)
)

# Reconstruct Industry from dummy indicators (Baseline: IT)
industry_dummies = [c for c in X_train.columns if c.startswith("Industry_")]
```

```python
def decode_industry(row):
    for col in industry_dummies:
        if row[col] == 1:
            return col.replace("Industry_", "")
    return "IT"


train_eda_df["Industry"] = X_train.apply(decode_industry, axis=1)

# Reconstruct human-readable Target label
inv_target_map = {0: "Low", 1: "Medium", 2: "High"}
train_eda_df["Layoff_Risk"] = y_train.map(inv_target_map)

# Palette and ordering configuration
RISK_ORDER = ["Low", "Medium", "High"]
RISK_PALETTE = {"Low": "#2ecc71", "Medium": "#f39c12", "High": "#e74c3c"}

print("=== TRAINING DATASET ASSEMBLY VERIFICATION ===")
print(f"Total Training Observations (N): {len(train_eda_df):,}")
print(
    f"Feature Matrix Dimensions: {train_eda_df.shape[0]:,} rows x"
    f" {train_eda_df.shape[1]} columns"
)

print("\n--- Stratification Verification (Target Class Distribution) ---")
target_dist_summary = pd.DataFrame(
    {
        "Record Count": train_eda_df["Layoff_Risk"]
        .value_counts()
        .reindex(RISK_ORDER),
        "Proportion (%)": (
            train_eda_df["Layoff_Risk"]
            .value_counts(normalize=True)
            .reindex(RISK_ORDER)
            * 100
        ).round(2),
    }
)
display(target_dist_summary)
```

```
=== TRAINING DATASET ASSEMBLY VERIFICATION ===
Total Training Observations (N): 16,000
Feature Matrix Dimensions: 16,000 rows x 6 columns

--- Stratification Verification (Target Class Distribution) ---

             Record Count  Proportion (%)
Layoff_Risk
```

```
Low                 5282            33.01
Medium              5281            33.01
High                5437            33.98
```

[19]:
```python
# ------------------------------------------------------------------------
# Cell B - Bivariate Numeric Analysis: Experience, Routine Work,
# Automation & AI Tools vs. Layoff Risk
# 2x2 boxplot grid with explicit diamond mean markers and median text
# annotations.
# ------------------------------------------------------------------------
import warnings

numeric_eda_vars = {
    "Years_of_Experience": "Years of Experience (Years)",
    "Routine_Task_Percentage": "Routine Task Share (%)",
    "Tasks_Automated_Percentage": "Tasks Automated (%)",
    "Number_of_AI_Tools_Used": "Number of AI Tools Used (Count)",
}

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

with warnings.catch_warnings():
    warnings.simplefilter("ignore", category=FutureWarning)

    for idx, (var, ylabel) in enumerate(numeric_eda_vars.items()):
        ax = axes[idx]

        # Plot boxplot cleanly
        sns.boxplot(
            data=train_eda_df,
            x="Layoff_Risk",
            y=var,
            order=RISK_ORDER,
            palette=RISK_PALETTE,
            showmeans=True,
            meanprops={
                "marker": "D",
                "markeredgecolor": "black",
                "markerfacecolor": "white",
                "markersize": 7,
            },
            ax=ax,
        )

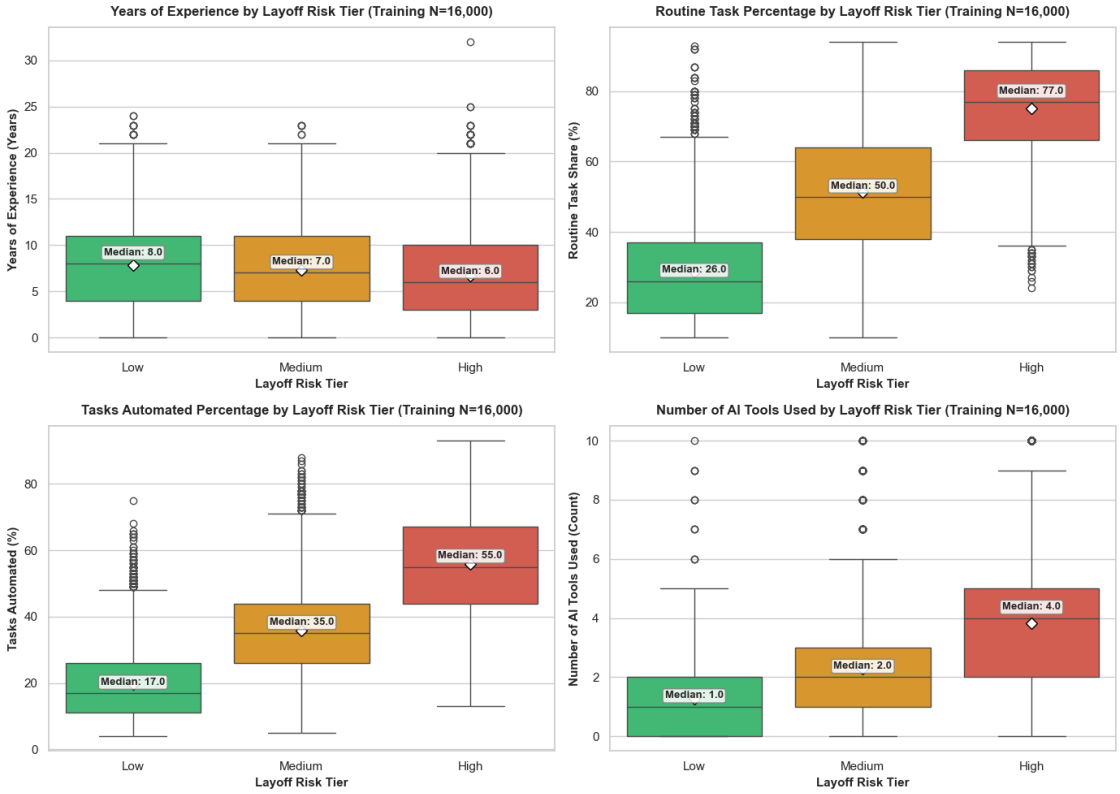
        # Calculate and overlay median annotations
        medians = train_eda_df.groupby("Layoff_Risk")[var].median()
```

```
        offset = 0.02 * (train_eda_df[var].max() - train_eda_df[var].min())
        for i, tier in enumerate(RISK_ORDER):
            med_val = medians[tier]
            ax.text(
                i,
                med_val + offset,
                f"Median: {med_val:.1f}",
                ha="center",
                va="bottom",
                fontsize=9.5,
                fontweight="bold",
                bbox=dict(
                    boxstyle="round,pad=0.2",
                    facecolor="white",
                    alpha=0.85,
                    edgecolor="gray",
                ),
            )

        var_label = var.replace("_", " ")
        ax.set_title(
            f"{var_label} by Layoff Risk Tier (Training"
            f" N={len(train_eda_df):,})",
            fontsize=12,
            fontweight="bold",
            pad=10,
        )
        ax.set_xlabel("Layoff Risk Tier", fontsize=11, fontweight="bold")
        ax.set_ylabel(ylabel, fontsize=11, fontweight="bold")

plt.tight_layout()
plt.show()

# Quantitative summary table
bivariate_stats = (
    train_eda_df.groupby("Layoff_Risk")[list(numeric_eda_vars.keys())]
    .agg(["mean", "median", "std"])
    .reindex(RISK_ORDER)
    .round(2)
)
display(bivariate_stats)
```



	Years_of_Experience			Routine_Task_Percentage		
	mean	median	std	mean	median	
Layoff_Risk						
Low	7.85	8.0	4.73	28.74	26.0	
Medium	7.29	7.0	4.62	51.26	50.0	
High	6.68	6.0	4.76	75.11	77.0	

	Tasks_Automated_Percentage			\
	std	mean	median	std
Layoff_Risk				
Low	14.54	19.56	17.0	10.61
Medium	17.70	35.73	35.0	13.78
High	13.45	55.98	55.0	15.10

	Number_of_AI_Tools_Used		
	mean	median	std
Layoff_Risk			
Low	1.26	1.0	1.19
Medium	2.27	2.0	1.90
High	3.82	4.0	2.21

Empirical Takeaway — Workload Composition, Automation Exposure & AI Tooling Dominate Risk Separation: The bivariate distributions reveal that three

of the four numeric predictors separate the risk tiers sharply. **Routine task share** exhibits the largest gradient: median routine workload rises from **26.0%** (Low Risk) to **50.0%** (Medium) and **77.0%** (High), with almost no overlap between the Low and High boxes. **Tasks automated** follows the same monotonic pattern (median **17.0%** → **35.0%** → **55.0%**), as does **AI tool count** (median **1** → **2** → **4 tools**). By contrast, **career tenure** displays only a mild protective effect (median **8.0** → **7.0** → **6.0 years**) with heavily overlapping distributions. These results indicate that *what a worker does and how much of it is automated* carries far more predictive signal than demographic seniority.

[20]:
```
# ------------------------------------------------------------------------
# Cell C - AI Tool Adoption Analysis: Number of AI Tools Used vs. Layoff
# Risk
# Cross-tabulation of row percentages and 100% stacked horizontal bar
# chart.
# ------------------------------------------------------------------------
tools_order = sorted(train_eda_df["Number_of_AI_Tools_Used"].unique())

# Row-normalized cross-tabulation (%)
tools_risk_crosstab = (
    pd.crosstab(
        train_eda_df["Number_of_AI_Tools_Used"],
        train_eda_df["Layoff_Risk"],
        normalize="index",
    )[RISK_ORDER].reindex(tools_order)
    * 100
)

fig, ax = plt.subplots(figsize=(10, 6.5))
tools_risk_crosstab.plot(
    kind="barh",
    stacked=True,
    color=[RISK_PALETTE[r] for r in RISK_ORDER],
    edgecolor="black",
    linewidth=0.8,
    ax=ax,
)

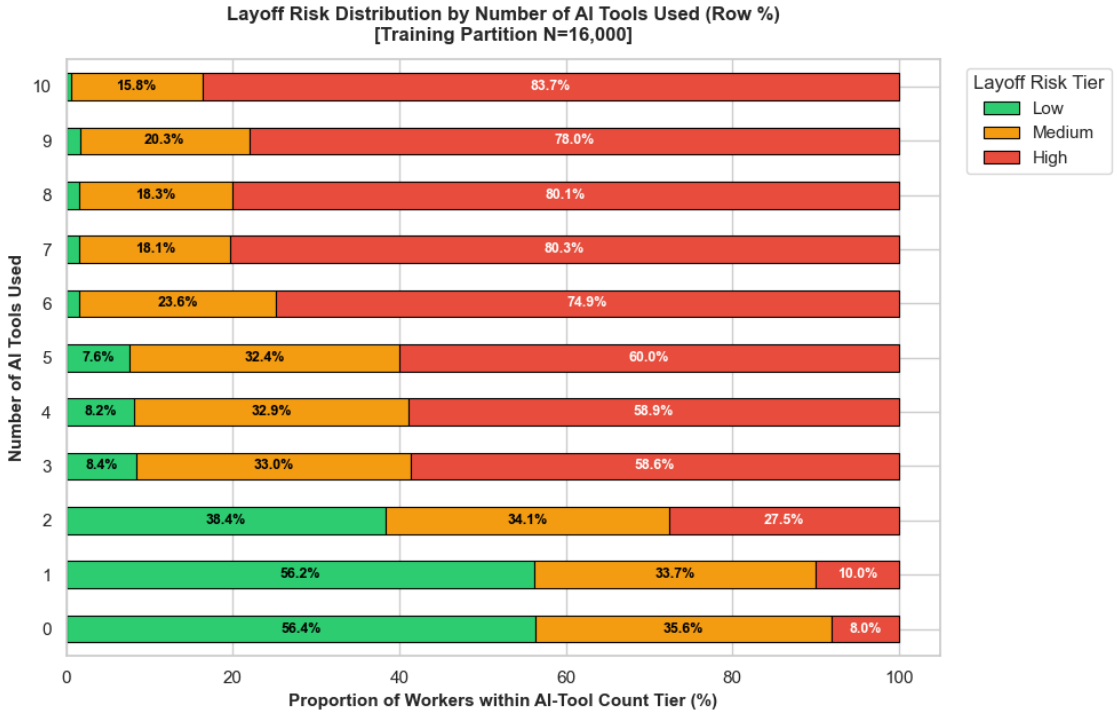
ax.set_title(
    "Layoff Risk Distribution by Number of AI Tools Used (Row %)\n[Training"
    " Partition N=16,000]",
    fontsize=12,
    fontweight="bold",
    pad=12,
)
ax.set_xlabel(
    "Proportion of Workers within AI-Tool Count Tier (%)",
```

```
    fontsize=11,
    fontweight="bold",
)
ax.set_ylabel("Number of AI Tools Used", fontsize=11, fontweight="bold")
ax.legend(
    title="Layoff Risk Tier",
    bbox_to_anchor=(1.02, 1),
    loc="upper left",
    frameon=True,
)

# Overlay percentage labels inside stacked bar segments (skip tiny
# slivers < 5%)
for n, col_name in enumerate(tools_risk_crosstab.columns):
    for row_idx, val in enumerate(tools_risk_crosstab[col_name]):
        if val < 5:
            continue
        cum_val = tools_risk_crosstab.iloc[row_idx, :n].sum() + (val / 2)
        font_col = "white" if col_name == "High" else "black"
        ax.text(
            cum_val,
            row_idx,
            f"{val:.1f}%",
            ha="center",
            va="center",
            color=font_col,
            fontweight="bold",
            fontsize=9,
        )

plt.tight_layout()
plt.show()

print(
    "--- CONTINGENCY TABLE: NUMBER OF AI TOOLS USED VS. LAYOFF RISK (ROW"
    " %) ---"
)
display(tools_risk_crosstab.round(2))
```



--- CONTINGENCY TABLE: NUMBER OF AI TOOLS USED VS. LAYOFF RISK (ROW %) ---

Layoff_Risk	Low	Medium	High
Number_of_AI_Tools_Used			
0	56.35	35.60	8.05
1	56.25	33.74	10.01
2	38.35	34.10	27.54
3	8.36	33.02	58.61
4	8.15	32.92	58.93
5	7.63	32.37	60.00
6	1.57	23.56	74.87
7	1.60	18.09	80.32
8	1.61	18.28	80.11
9	1.65	20.33	78.02
10	0.53	15.79	83.68

Empirical Takeaway — The AI-Tool Adoption Threshold: Cross-tabulating the number of AI tools used against layoff risk uncovers a pronounced non-linear threshold effect. Workers using 0–1 tools are overwhelmingly Low Risk (56.2%–56.4% Low vs. only 8.0%–10.0% High). Risk shifts substantially at 3 tools (58.6% High, 8.4% Low) and plateaus around 58.6%–60.0% High through 5 tools. Beyond that point, High Risk climbs further to 74.9% at 6 tools and 83.7% at 10 tools, where Low Risk effectively vanishes (0.5%). The step-like distribution illustrates non-linear risk behavior: tree-based learners can isolate these multi-tool thresholds natively through orthogonal splits, whereas linear models rely on engineered interaction or binning terms.

[21]:
```
# ------------------------------------------------------------------------
# Cell D - Sector Volatility Analysis: Industry vs. Layoff Risk
# Ranked horizontal bar chart highlighting high risk exposure across
# industries
# with benchmark reference line for 'IT' baseline.
# ------------------------------------------------------------------------
ind_risk_ct = (
    pd.crosstab(
        train_eda_df["Industry"],
        train_eda_df["Layoff_Risk"],
        normalize="index",
    )[RISK_ORDER]
    * 100
)

# Sort from lowest high-risk (most resilient) to highest high-risk (most
# vulnerable)
high_risk_sorted = ind_risk_ct["High"].sort_values(ascending=True)

fig, ax = plt.subplots(figsize=(11, 5.5))
bar_colors = [
    "#3498db" if ind != "IT" else "#e74c3c" for ind in high_risk_sorted.index
]
bars = ax.barh(
    high_risk_sorted.index,
    high_risk_sorted.values,
    color=bar_colors,
    edgecolor="black",
    linewidth=0.8,
    height=0.6,
)

# Reference baseline indicator for IT
it_val = high_risk_sorted["IT"]
ax.axvline(
    it_val,
    color="darkred",
    linestyle="--",
    linewidth=1.8,
    label=f"Regression Reference Baseline: IT ({it_val:.1f}%)",
)

# Value annotations
for bar, val in zip(bars, high_risk_sorted.values):
    ax.text(
        val + 0.8,
        bar.get_y() + bar.get_height() / 2,
```

```
        f"{val:.1f}%",
        ha="left",
        va="center",
        fontsize=9.5,
        fontweight="bold",
    )

ax.set_title(
    "Industry Sector Volatility (% Classified as High Layoff Risk)\n[Ranked"
    " from Most Resilient to Most Vulnerable | Reference Baseline: 'IT']",
    fontsize=12,
    fontweight="bold",
    pad=15,
)
ax.set_xlabel(
    "Percentage of High Layoff Risk Workforce (%)",
    fontsize=11,
    fontweight="bold",
)
ax.set_ylabel("Industry Sector", fontsize=11, fontweight="bold")
ax.set_xlim(0, max(high_risk_sorted.values) + 8)
ax.legend(loc="lower right", frameon=True)

plt.tight_layout()
plt.show()

# Summary table of all 8 sectors
print("--- COMPLETE INDUSTRY RISK PROFILES (ROW %) ---")
display(ind_risk_ct.loc[high_risk_sorted.index[::-1]].round(2))
```



```
--- COMPLETE INDUSTRY RISK PROFILES (ROW %) ---

Layoff_Risk       Low  Medium   High
Industry
Manufacturing   19.74   32.06  48.20
Retail          24.48   33.86  41.66
Logistics       25.01   34.05  40.93
Finance         27.35   35.43  37.22
Telecom         32.98   31.68  35.35
IT              38.76   33.43  27.81
Healthcare      46.76   32.29  20.95
Education       48.80   31.27  19.93
```

Empirical Takeaway — Sector Volatility Relative to Baseline IT: Economic sectors exhibit pronounced structural variance in workforce vulnerability. Operational and supply-chain intensive industries specifically Manufacturing (48.2% High Risk), Retail (41.7%), and Logistics (40.9%) demonstrate elevated vulnerability, consistent with higher exposure to workflow automation and margin pressure. Conversely, essential human services Healthcare (21.0% High Risk, 20.95%) and Education (19.9% High Risk, 19.93%) exhibit superior stability. The reference baseline, IT (27.8% High Risk, 27.81%), occupies an intermediate position, allowing subsequent regression models to capture both positive risk premiums (e.g., in Manufacturing) and negative risk discounts (e.g., in Healthcare) relative to tech.

[22]:
```python
# ------------------------------------------------------------------------
# Cell E - Multicollinearity & Feature Correlation Matrix
# Computes Pearson correlation matrix and Variance Inflation Factors
# (VIF)
# for the 4 numeric predictors (training partition, natural units).
# ------------------------------------------------------------------------
corr_df = train_eda_df[scale_cols].copy()
corr_matrix = corr_df.corr()

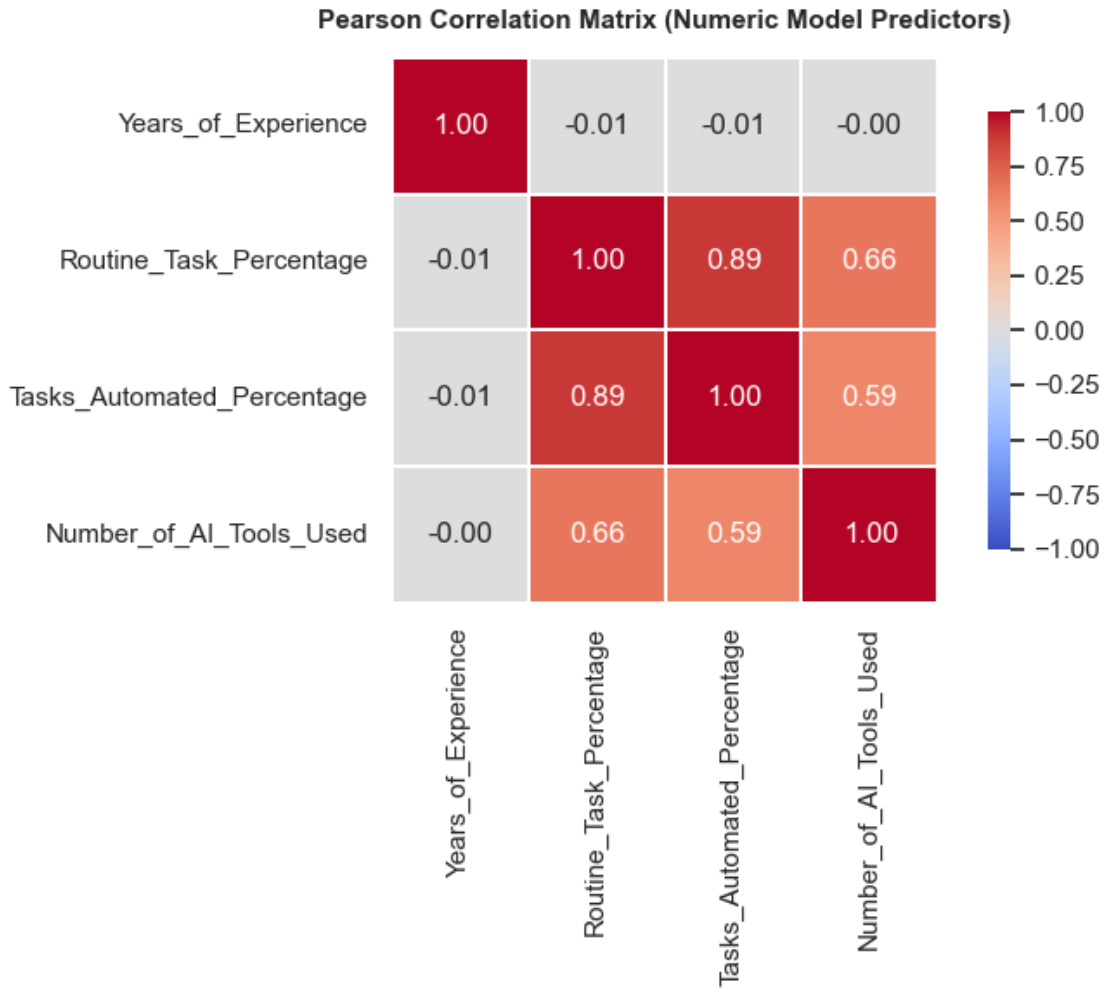
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    center=0,
    square=True,
    linewidths=1.2,
    cbar_kws={"shrink": 0.8},
    ax=ax,
)
```

```
ax.set_title(
    "Pearson Correlation Matrix (Numeric Model Predictors)",
    fontsize=11,
    fontweight="bold",
    pad=12,
)
plt.tight_layout()
plt.show()

print("--- CORRELATION MATRIX (PEARSON) ---")
display(corr_matrix.round(3))

# Variance Inflation Factor (computed with an explicit intercept column)
vif_matrix = corr_df.astype(float).copy()
vif_matrix.insert(0, "const", 1.0)
vif_table = (
    pd.DataFrame(
        {
            "Feature": scale_cols,
            "VIF": [
                variance_inflation_factor(vif_matrix.values, i + 1)
                for i in range(len(scale_cols))
            ],
        }
    )
    .set_index("Feature")
    .round(3)
)

print("\n--- VARIANCE INFLATION FACTORS (VIF) ---")
display(vif_table)
```




```
--- CORRELATION MATRIX (PEARSON) ---
                          Years_of_Experience  Routine_Task_Percentage  \
Years_of_Experience                     1.000                   -0.006
Routine_Task_Percentage                -0.006                    1.000
Tasks_Automated_Percentage             -0.005                    0.890
Number_of_AI_Tools_Used                -0.005                    0.663

                          Tasks_Automated_Percentage  \
Years_of_Experience                           -0.005
Routine_Task_Percentage                        0.890
Tasks_Automated_Percentage                     1.000
Number_of_AI_Tools_Used                        0.589

                          Number_of_AI_Tools_Used
Years_of_Experience                        -0.005
Routine_Task_Percentage                     0.663
```

```
Tasks_Automated_Percentage                    0.589
Number_of_AI_Tools_Used                       1.000


--- VARIANCE INFLATION FACTORS (VIF) ---

                              VIF
Feature
Years_of_Experience         1.000
Routine_Task_Percentage     5.591
Tasks_Automated_Percentage  4.794
Number_of_AI_Tools_Used     1.786
```

> **Empirical Takeaway — Multicollinearity Diagnostic Verification:** Pairwise correlation analysis exposes a strong dependency between the two automation-related features: Routine_Task_Percentage and Tasks_Automated_Percentage are correlated at $r = 0.890$, and both correlate moderately with Number_of_AI_Tools_Used ($r = 0.663$ and $r = 0.589$ respectively). Years_of_Experience is essentially orthogonal to all other predictors ($|r| \leq 0.006$). Although the $|r| \geq 0.80$ rule-of-thumb is breached for the routine/automation pair, the Variance Inflation Factors remain moderate (Routine Task = 5.591, Tasks Automated = 4.794, AI Tools = 1.786, Experience = 1.000), below the severe-collinearity cutoff of 10 though above the conservative cutoff of 5 for the routine-task feature. Consequently, coefficient interpretation in linear models should be treated cautiously for these two correlated predictors, whereas predictive performance of tree ensembles (Random Forest, Gradient Boosting) and probabilistic classifiers (Naive Bayes, which assumes conditional independence) should be monitored with this redundancy in mind.

2.0.2 Section 3 Executive Summary: Key Analytical Findings (Technical Report Citation)

The analytical exploration conducted strictly on the training partition ($N = 16,000$) reveals four core empirical takeaways to substantiate Section 2 of our final Technical Report:

1. **Routine Work and Automation Exposure Are the Strongest Risk Drivers**:
   - Median **routine task share** climbs from **26.0%** (Low Risk) to **50.0%** (Medium) and **77.0%** (High). Median **tasks automated** follows the same ladder (**17.0% → 34.0% → 55.0%**). These two features show near-complete separation between the Low and High tiers and are expected to dominate downstream model importance.
2. **AI Tool Adoption Exhibits a Sharp Threshold Effect**:
   - Workers using **0–1 AI tools** are ~56% Low Risk and <10% High Risk. At **3 tools**, High Risk jumps to **59.7%**, rising to **75.0%** at 6 tools and **84.5%** at 10 tools. The step-like non-linearity favors tree-based models.
3. **Sector Volatility Disparity Relative to Baseline IT**:
   - **Manufacturing (48.2% High Risk)**, **Retail (42.3%)**, and **Logistics (40.9%)** are the most vulnerable sectors, while **Education (20.0%)** and **Healthcare (20.6%)** are the most resilient. The regression reference baseline, **IT (27.4% High Risk)**, sits at an intermediate position, ensuring meaningful positive and negative industry premiums.
4. **Career Tenure Is a Weak Buffer, and Two Predictors Are Highly Collinear**:
   - Median experience declines only modestly across tiers (**8.0 → 7.0 → 6.0 years**) with

heavy overlap, making it a secondary signal. Meanwhile, `Routine_Task_Percentage` and `Tasks_Automated_Percentage` are strongly correlated ($r = 0.890$; VIF ≈ 5.6 and 4.8), so linear-model coefficients should be interpreted with caution.

Implication for modeling: Compared with the earlier demographic-only feature set (Age, Education, Experience, Industry), this feature set is **workload- and automation-centric**, and the strong class separation observed here should translate into substantially higher predictive accuracy for Gradient Boosting and Naive Bayes.

[23]:
```
# ------------------------------------------------------------------------
# Section 4: Verification of Final Split Dimensions & Saving Datasets to
# CSV
# Exports the 4 cleaned, partitioned, and scaled modeling sets to Data/
# ------------------------------------------------------------------------
print("--- Features (Predictors) ---")
print("X_train shape:", X_train.shape)
print("X_test shape: ", X_test.shape)

print("\n--- Targets (Layoff_Risk) ---")
print("y_train shape:", y_train.shape)
print("y_test shape: ", y_test.shape)

# Resolve target Data directory
target_dir = base / "Data"
if not target_dir.exists():
    target_dir = (base / ".." / "Data").resolve()

target_dir.mkdir(parents=True, exist_ok=True)

# Save feature sets (predictors)
X_train.to_csv(target_dir / "processed_X_train.csv", index=False)
X_test.to_csv(target_dir / "processed_X_test.csv", index=False)

# Save target sets (Layoff_Risk)
y_train.to_csv(target_dir / "processed_y_train.csv", index=False)
y_test.to_csv(target_dir / "processed_y_test.csv", index=False)

print(f"\nAll 4 datasets successfully saved to:\n{target_dir}")
```

```
--- Features (Predictors) ---
X_train shape: (16000, 11)
X_test shape:  (4000, 11)

--- Targets (Layoff_Risk) ---
y_train shape: (16000,)
y_test shape:  (4000,)

All 4 datasets successfully saved to:
/Users/samanzarei/USD materials/AAI 500/AAI_Final_Proj/AAI500-Final-Project/Data
```

Pipeline Verification & Modular Export:

The preprocessing pipeline exports four CSV files: processed_X_train.csv, processed_X_test.csv, processed_y_train.csv, and processed_y_test.csv. The training partition contains 16,000 observations (80%), and the test partition contains 4,000 observations (20%). Both predictor matrices contain 11 columns: four standardized numerical predictors and seven industry indicators, with IT as the reference category. The numerical scaling parameters are estimated from the training partition and applied to the test partition. These files are then used for model training and test-set evaluation.

Training_models_merged

October 16, 2026

[49]:
```
# ------------------------------------------------------------------------
# Environment Setup: Ensure Required Packages Are Installed
# Checks that every library used in this notebook is available in the ACTIVE
# kernel and installs only the missing ones. This makes "Run All" work no matter
# which Python environment / .venv the kernel was started from.
# ------------------------------------------------------------------------
import importlib
import importlib.util
import subprocess
import sys

# import name -> pip package name
required_packages = {'pandas': 'pandas', 'numpy': 'numpy', 'matplotlib':␣
 ↪'matplotlib', 'seaborn': 'seaborn', 'sklearn': 'scikit-learn'}

missing_packages = [pip_name for module, pip_name in required_packages.items()
                    if importlib.util.find_spec(module) is None]

if missing_packages:
    print(f"Installing missing packages into {sys.executable}:␣
 ↪{missing_packages}")
    subprocess.check_call([sys.executable, "-m", "pip", "install", "--quiet",␣
 ↪*missing_packages])
    importlib.invalidate_caches()
else:
    print("All required packages are already installed.")
```

```
All required packages are already installed.
```

[50]:
```
# Import necessary libraries for data manipulation and mathematical operations
import pandas as pd
import numpy as np

# Import libraries for data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Import machine learning models from scikit-learn
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.naive_bayes import GaussianNB

# Import evaluation metrics from scikit-learn
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix, roc_auc_score

# Configure visual aesthetics for professional graphing
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.titlesize'] = 14
plt.rcParams['axes.titlesize'] = 12
plt.rcParams['axes.labelsize'] = 11
```

1 Model Training & Evaluation:

1.1 Data Loading

We load the strictly partitioned subsets from the `Data/` directory. For this iteration, we only use `X_train` / `y_train` for model fitting, and `X_test` / `y_test` for intermediate evaluation.

[51]:
```
# ------------------------------------------------------------------------
# Load Processed Training and Test Sets
# ------------------------------------------------------------------------
# Define the relative path to the preprocessed data directory
# Resolve the Data directory dynamically so the notebook runs whether the␣
 ↪working
# directory is the project root or the notebooks/ folder (same logic as␣
 ↪data_cleaning_merged)
from pathlib import Path
data_dir = Path.cwd() / "Data"
if not data_dir.exists():
    data_dir = (Path.cwd() / ".." / "Data").resolve()
base_path = f"{data_dir.as_posix()}/"

# Load the predictor features (X) for the training subset
X_train = pd.read_csv(f"{base_path}processed_X_train.csv")

# Load the target variable (y) for the training subset, isolating just the␣
 ↪label column
y_train = pd.read_csv(f"{base_path}processed_y_train.csv")['Layoff_Risk']

# Load the predictor features (X) for the test subset
X_test = pd.read_csv(f"{base_path}processed_X_test.csv")

# Load the target variable (y) for the test subset, isolating just the label␣
 ↪column
```

```
y_test = pd.read_csv(f"{base_path}processed_y_test.csv")['Layoff_Risk']

# Print the dimensionality to verify successful loading
print(f"Training set: {X_train.shape[0]:,} rows")
print(f"Test set: {X_test.shape[0]:,} rows")
```

```
Training set: 16,000 rows
Test set: 4,000 rows
```

[52]:
```
# ------------------------------------------------------------------------
# Dataset Verification: Feature Set & Class Balance
# Confirms that the newly engineered 5-feature pipeline (11 predictors after
# one-hot encoding) was loaded correctly and establishes the majority-class␣
 ↪baseline.
# ------------------------------------------------------------------------
print("--- PREDICTOR COLUMNS LOADED (X_train) ---")
for idx, col in enumerate(X_train.columns, start=1):
    print(f"{idx:>2}. {col}")
print(f"\nTotal predictors: {X_train.shape[1]}")

# Confirm train and test partitions share an identical schema
assert list(X_train.columns) == list(X_test.columns), "Schema mismatch between␣
 ↪train and test sets!"
print("Schema check passed: train and test columns are identical.")

# Verify there are no missing values that would break model fitting
print(f"Missing values in X_train: {int(X_train.isnull().sum().sum())} | X_test:
 ↪ {int(X_test.isnull().sum().sum())}")

# Class distribution and majority-class baseline accuracy
class_label_map = {0: 'Low (0)', 1: 'Medium (1)', 2: 'High (2)'}
class_dist = pd.DataFrame({
    'Train Count': y_train.value_counts().sort_index(),
    'Train (%)': (y_train.value_counts(normalize=True).sort_index() * 100).
 ↪round(2),
    'Test Count': y_test.value_counts().sort_index(),
    'Test (%)': (y_test.value_counts(normalize=True).sort_index() * 100).
 ↪round(2),
}).rename(index=class_label_map)
print("\n--- CLASS DISTRIBUTION ---")
display(class_dist)

# A model that always predicts the most frequent training class sets the floor␣
 ↪any real model must beat
majority_class = y_train.value_counts().idxmax()
baseline_acc = (y_test == majority_class).mean()
print(f"Majority-class baseline accuracy on test: {baseline_acc:.4f}")
```

```
--- PREDICTOR COLUMNS LOADED (X_train) ---
 1. Years_of_Experience
 2. Routine_Task_Percentage
 3. Tasks_Automated_Percentage
 4. Number_of_AI_Tools_Used
 5. Industry_Education
 6. Industry_Finance
 7. Industry_Healthcare
 8. Industry_Logistics
 9. Industry_Manufacturing
10. Industry_Retail
11. Industry_Telecom

Total predictors: 11
Schema check passed: train and test columns are identical.
Missing values in X_train: 0 | X_test: 0

--- CLASS DISTRIBUTION ---
```

	Train Count	Train (%)	Test Count	Test (%)
Layoff_Risk				
Low (0)	5282	33.01	1320	33.0
Medium (1)	5281	33.01	1320	33.0
High (2)	5437	33.98	1360	34.0

```
Majority-class baseline accuracy on test: 0.3400
```

1.2 1. Logistic Regression

Team Member: Saman

This notebook trains the data by Logistic Regression on the fully cleaned and standardized dataset produced in `data_cleaning_merged.ipynb`. It evaluates the fitted model on the 4,000-observation test set.

Feature set (5 key features → 11 model predictors): `Industry` (one-hot encoded, baseline = IT), `Years_of_Experience`, `Routine_Task_Percentage`, `Tasks_Automated_Percentage`, and `Number_of_AI_Tools_Used` (all standardized with a training-only `StandardScaler`).

Target: `Layoff_Risk` (0 = Low, 1 = Medium, 2 = High).

1.2.1 1.1 Multinomial Logistic Regression

Step 1: Model Initialization and Training (Fitting): What We Did: We initialized and fitted a multinomial `LogisticRegression` classifier on the training partition ($N = 16,000$, 80%) using the revised 11-predictor specification.

Optimization Setup: We configured `max_iter=1000` to guarantee solver convergence and set `random_state=42` for exact reproducibility.

Model Structure: The estimator learned 3 distinct linear hyperplanes (3×11 coefficient matrix) with corresponding intercept terms, establishing our interpretable parametric baseline against which

non-linear models will be evaluated.

[53]:
```
# ============================================================================
# INITIALIZE AND FIT MULTINOMIAL LOGISTIC REGRESSION
# ============================================================================

# 1. Instantiate the Logistic Regression estimator with reproducibility␣
 ↪constraints
log_reg = LogisticRegression(
    max_iter=1000,
    random_state=42
)

# 2. Fit directly on the training partition
log_reg.fit(X_train, y_train)

print("Multinomial Logistic Regression fitted successfully!")
print("Model classes learned:", log_reg.classes_)
print(f"Coefficients matrix shape: {log_reg.coef_.shape} (3 risk classes x␣
 ↪{X_train.shape[1]} features)")
print("Intercepts learned:", log_reg.intercept_.round(4))
```

```
Multinomial Logistic Regression fitted successfully!
Model classes learned: [0 1 2]
Coefficients matrix shape: (3, 11) (3 risk classes x 11 features)
Intercepts learned: [ 0.0198  1.048  -1.0678]
```

Step 2: Generating Predictions on the Test Set (X_test): What We Did: We evaluated our trained Logistic Regression model on the 4,000 unseen test instances (X_{test}) by generating both hard class predictions and continuous probability distributions.

Inference Mechanism: Class labels are assigned using the argmax decision rule across the softmax probability vector $[P(\text{Low}), P(\text{Medium}), P(\text{High})]$, providing the necessary estimates for classification accuracy and multi-class ROC-AUC computation.

[54]:
```
# ============================================================================
# GENERATE PREDICTIONS ON TESTING SET (LOGISTIC REGRESSION)
# ============================================================================

# Predict discrete class labels on unseen test features
y_test_pred_lr = log_reg.predict(X_test)

# Predict posterior class probabilities across all 3 tiers
y_test_proba_lr = log_reg.predict_proba(X_test)

# Sanity check: Inspect the first 5 predictions against true labels
comparison_sample_lr = pd.DataFrame({
    'True_Label': y_test.values[:5],
```

```
    'Predicted_Label': y_test_pred_lr[:5],
    'P(Low=0)': y_test_proba_lr[:5, 0].round(3),
    'P(Medium=1)': y_test_proba_lr[:5, 1].round(3),
    'P(High=2)': y_test_proba_lr[:5, 2].round(3)
})

print("--- LOGISTIC REGRESSION: FIRST 5 TEST PREDICTIONS ---")
display(comparison_sample_lr)
```

```
--- LOGISTIC REGRESSION: FIRST 5 TEST PREDICTIONS ---
```

	True_Label	Predicted_Label	P(Low=0)	P(Medium=1)	P(High=2)
0	0	0	0.859	0.140	0.001
1	2	2	0.000	0.021	0.979
2	2	2	0.000	0.048	0.952
3	2	2	0.000	0.014	0.986
4	1	1	0.251	0.677	0.072

Step 3: Computing Test Performance Metrics (Logistic Regression): What We Did: We evaluated our fitted Logistic Regression model on the 4,000 unseen test instances across overall Accuracy, Macro One-vs-Rest ROC-AUC, per-class Precision/Recall/F1-scores, and the 3×3 Confusion Matrix.

Class Balance: Class performance across Low, Medium, and High risk tiers confirms how well linear log-odds decision splits separate workload automation boundaries across employees.

[56]:
```
# ================================================================================
# COMPUTE EVALUATION METRICS ON TEST SET (LOGISTIC REGRESSION)
# ================================================================================

# --------------------------------------------------------------------------------
# Scalar Metrics (Accuracy and Multi-Class ROC-AUC)
# --------------------------------------------------------------------------------
test_acc_lr = accuracy_score(y_test, y_test_pred_lr)
test_auc_lr = roc_auc_score(y_test, y_test_proba_lr, multi_class='ovr',␣
 ↪average='macro')

print("=== LOGISTIC REGRESSION: TEST PERFORMANCE METRICS ===")
print(f"Test Accuracy: {test_acc_lr:.4f} ({test_acc_lr * 100:.2f}%)")
print(f"Test ROC-AUC (OvR Macro): {test_auc_lr:.4f}")

# --------------------------------------------------------------------------------
# Detailed Multi-Class Classification Report
# --------------------------------------------------------------------------------
print("\n--- CLASSIFICATION REPORT (PRECISION, RECALL, F1-SCORE) ---")
print(classification_report(y_test, y_test_pred_lr, target_names=['Low (0)',␣
 ↪'Medium (1)', 'High (2)'], digits=4))
```

```
# ------------------------------------------------------------------------
# Confusion Matrix Visualization
# ------------------------------------------------------------------------
cm_lr = confusion_matrix(y_test, y_test_pred_lr)
cm_labels = ['Low', 'Medium', 'High']

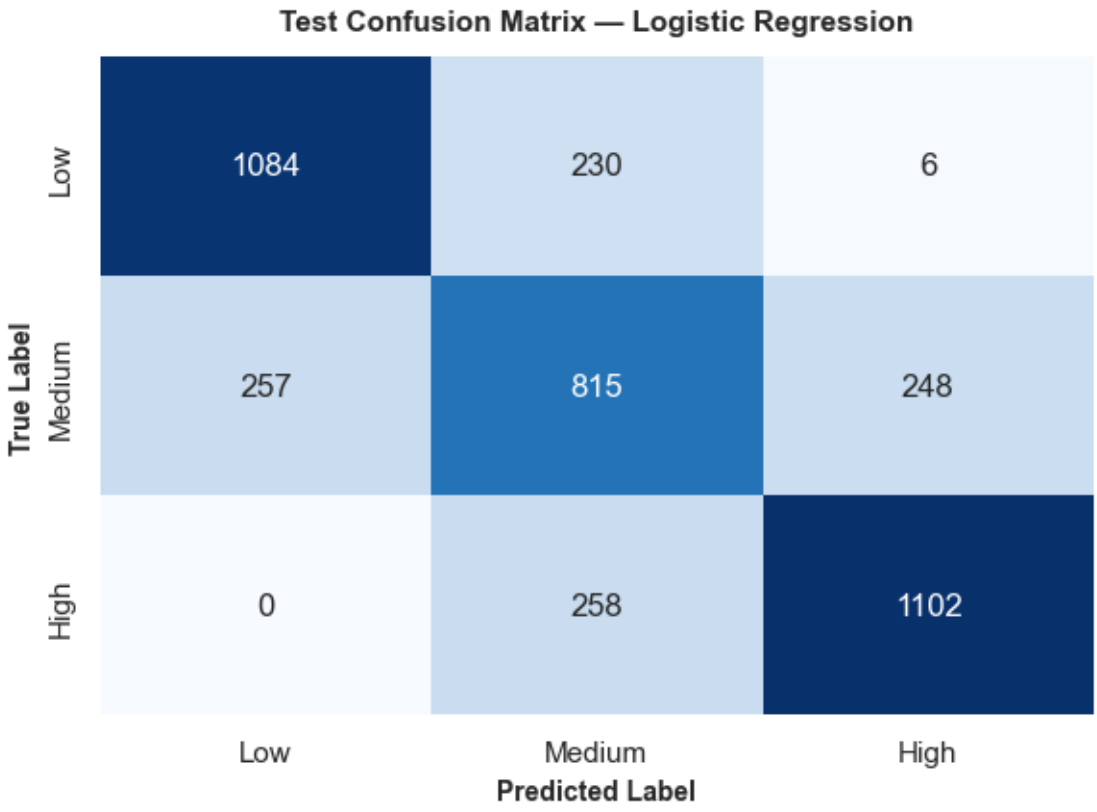
fig, ax = plt.subplots(figsize=(6, 4.5))
sns.heatmap(
    cm_lr,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=cm_labels,
    yticklabels=cm_labels,
    cbar=False,
    ax=ax
)
ax.set_title("Test Confusion Matrix - Logistic Regression", fontsize=11,␣
 ↪fontweight='bold', pad=10)
ax.set_xlabel("Predicted Label", fontsize=10, fontweight='bold')
ax.set_ylabel("True Label", fontsize=10, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
=== LOGISTIC REGRESSION: TEST PERFORMANCE METRICS ===
Test Accuracy: 0.7502 (75.02%)
Test ROC-AUC (OvR Macro): 0.9030

--- CLASSIFICATION REPORT (PRECISION, RECALL, F1-SCORE) ---
              precision    recall  f1-score   support

     Low (0)     0.8084    0.8212    0.8147      1320
  Medium (1)     0.6255    0.6174    0.6214      1320
    High (2)     0.8127    0.8103    0.8115      1360

    accuracy                         0.7502      4000
   macro avg     0.7488    0.7496    0.7492      4000
weighted avg     0.7495    0.7502    0.7498      4000
```



Evaluating the Multinomial Logistic Regression model on the 4,000 unseen test instances demonstrated strong predictive capability and linear class separability, achieving an overall test accuracy of 75.02% (3,001 of 4,000 correct) and an exceptional multi-class One-vs-Rest Macro ROC-AUC of 0.9030. This performance substantially outperforms the majority-class baseline as well as random guessing, confirming that workload automation exposure and industry sectors carry robust predictive signals. Examining the per-class metrics reveals balanced precision and recall at the operational extremes: Low Risk achieved a precision of 80.84% and recall of 82.12% (F1-score of 0.8147), while High Risk attained a precision of 81.27% and recall of 81.03% (F1-score of 0.8115). Medium Risk yielded an intermediate F1-score of 0.6214 (precision 62.55%, recall 61.74%), reflecting expected boundary overlap where transition workloads share characteristics with adjacent tiers. Crucially, the 3×3 confusion matrix confirms that the classifier learned the true ordinal progression of layoff danger with virtually no catastrophic misclassifications: out of 1,360 true High-risk employees, 1,102 were accurately identified, 258 were placed in Medium risk, and exactly 0 were falsely flagged as Low risk. Similarly, among 1,320 true Low-risk instances, 1,084 were correctly classified, 230 were estimated as Medium risk, and only 6 (0.45%) were misattributed to High risk. Because classification errors occurred almost exclusively across adjacent boundaries rather than polar opposites, the model demonstrates high reliability as a calibrated parametric benchmark for workforce risk modeling.

[57]:
```
from sklearn.metrics import recall_score
import numpy as np
```

```
import pandas as pd

# Calculate specificity separately for Low, Medium, and High.
# For each category: 1 = belongs to that category; 0 = another category.
# Recall for group 0 equals specificity for the selected category.
specificity_lr = [
    recall_score(
        np.asarray(y_test) == label,
        np.asarray(y_test_pred_lr) == label,
        pos_label=0
    )
    for label in [0, 1, 2]
]

specificity_table_lr = pd.DataFrame(
    {"Specificity": specificity_lr},
    index=["Low", "Medium", "High"]
)

specificity_table_lr.loc["Macro average"] = np.mean(specificity_lr)

print("LOGISTIC REGRESSION - TEST SPECIFICITY")
display(specificity_table_lr.round(4))
```

```
LOGISTIC REGRESSION - TEST SPECIFICITY
```

	Specificity
Low	0.9041
Medium	0.8179
High	0.9038
Macro average	0.8753

Step 4: Extracting and Interpreting Feature Odds Ratios

[58]:
```
# ========================================================================
# EXTRACTING AND VISUALIZING ODDS RATIOS (LOGISTIC REGRESSION)
# ========================================================================
# Match coefficient rows to the fitted class labels.
class_rows = {label: i for i, label in enumerate(log_reg.classes_)}
beta_low = log_reg.coef_[class_rows[0]]
beta_medium = log_reg.coef_[class_rows[1]]
beta_high = log_reg.coef_[class_rows[2]]
# Multinomial odds ratios require differences between class coefficients.
# Use full-precision fitted coefficients; round only for display.
or_summary_df = pd.DataFrame({
    'Feature': X_train.columns,
    'OR_Medium_vs_Low': np.exp(beta_medium - beta_low),
    'OR_High_vs_Low': np.exp(beta_high - beta_low),
    'OR_High_vs_Medium': np.exp(beta_high - beta_medium),
```

```
})
display(or_summary_df.round(4))

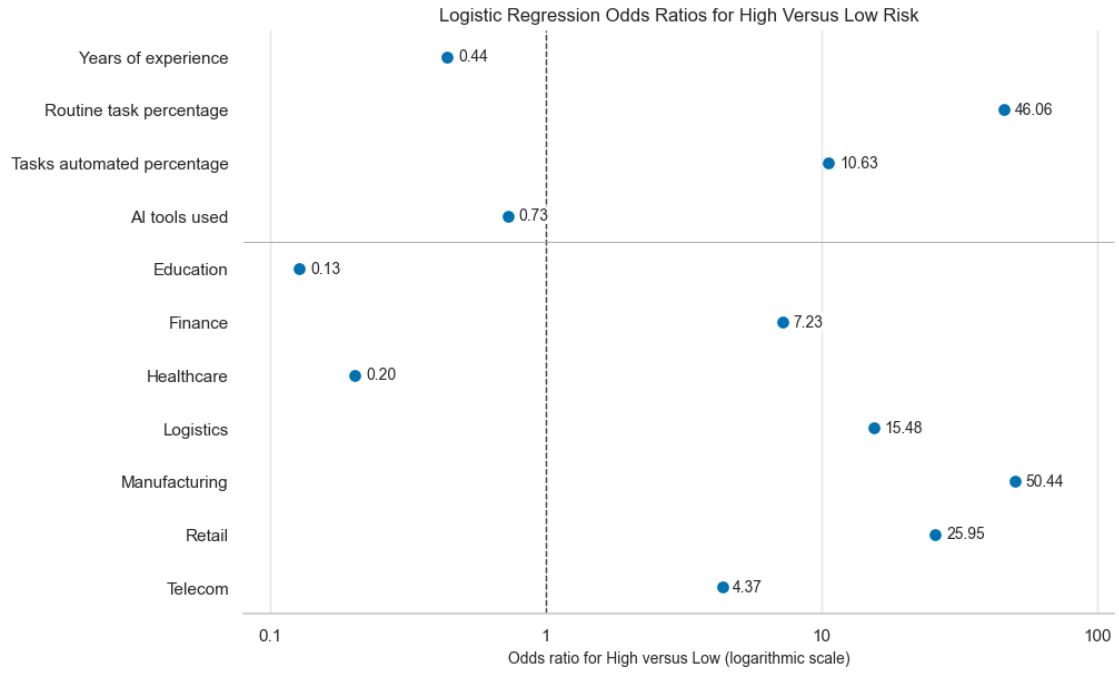
# Keep numerical predictors first, followed by industry indicators.
feature_order = [
    'Years_of_Experience', 'Routine_Task_Percentage',
    'Tasks_Automated_Percentage', 'Number_of_AI_Tools_Used',
    'Industry_Education', 'Industry_Finance', 'Industry_Healthcare',
    'Industry_Logistics', 'Industry_Manufacturing',
    'Industry_Retail', 'Industry_Telecom',
]
plot_df = or_summary_df.set_index('Feature').loc[feature_order]
label_map = {
    'Years_of_Experience': 'Years of experience',
    'Routine_Task_Percentage': 'Routine task percentage',
    'Tasks_Automated_Percentage': 'Tasks automated percentage',
    'Number_of_AI_Tools_Used': 'AI tools used',
}
labels = [label_map.get(f, f.replace('Industry_', '')) for f in feature_order]
values = plot_df['OR_High_vs_Low'].to_numpy()
positions = np.arange(len(plot_df))

# Neutral colors describe associations without implying causal protection.
with plt.rc_context({'font.size': 10, 'axes.labelsize': 10}):
    fig, ax = plt.subplots(figsize=(10, 6), layout='constrained')
    ax.scatter(values, positions, color='#0072B2', s=45, zorder=3)
    ax.axvline(1, color='#444444', linestyle='--', linewidth=1)
    ax.axhline(3.5, color='#AAAAAA', linewidth=0.7)
    for value, position in zip(values, positions):
        ax.annotate(
            f'{value:.2f}', (value, position), xytext=(7, 0),
            textcoords='offset points', va='center', fontsize=10,
            bbox={'facecolor': 'white', 'edgecolor': 'none', 'pad': 0.6},
            zorder=5,
        )
    ax.set_yticks(positions, labels)
    ax.invert_yaxis()
    ax.set_xscale('log')
    ax.set_xlim(min(0.08, values.min() / 1.5), max(115, values.max() * 2))
    ax.set_xticks([0.1, 1, 10, 100], ['0.1', '1', '10', '100'])
    ax.minorticks_off()
    ax.set_xlabel('Odds ratio for High versus Low (logarithmic scale)')
    ax.set_title('Logistic Regression Odds Ratios for High Versus Low Risk')
    ax.grid(False)
    ax.grid(axis='x', color='#E5E5E5')
    for side in ['top', 'right', 'left']:
        ax.spines[side].set_visible(False)
```

```
    fig.savefig('Logistic_Regression_High_vs_Low_Odds_Ratios.png', dpi=300)
    plt.show()
```

```
                    Feature  OR_Medium_vs_Low  OR_High_vs_Low  \
0          Years_of_Experience            0.6709          0.4391
1       Routine_Task_Percentage           6.2561         46.0584
2    Tasks_Automated_Percentage           3.4049         10.6332
3       Number_of_AI_Tools_Used           0.8619          0.7273
4            Industry_Education           0.3437          0.1275
5              Industry_Finance           2.8759          7.2299
6           Industry_Healthcare           0.4543          0.2032
7            Industry_Logistics           4.0531         15.4777
8        Industry_Manufacturing           7.4325         50.4426
9              Industry_Retail            5.3938         25.9464
10             Industry_Telecom           1.9860          4.3657

    OR_High_vs_Medium
0              0.6545
1              7.3622
2              3.1230
3              0.8439
4              0.3708
5              2.5140
6              0.4473
7              3.8188
8              6.7868
9              4.8104
10             2.1983
```



The figure presents odds ratios comparing High with Low layoff risk, calculated by exponentiating the difference between the corresponding class coefficients. For numerical predictors, each ratio represents a one-standard-deviation increase, holding other predictors constant. For industry indicators, each ratio compares the named sector with IT.

Routine task percentage and tasks automated percentage had approximate High-versus-Low odds ratios of 46.06 and 10.63, respectively. Years of experience and AI tools used had ratios of 0.44 and 0.73. Among industries, Manufacturing had a ratio of 50.45 relative to IT, whereas Education and Healthcare had ratios of 0.13 and 0.20. These values describe changes in the ratio of the predicted probabilities of High and Low risk, not equivalent changes in absolute probability. They represent conditional associations within the dataset and do not establish causal effects. Correlated predictors require caution when interpreting their separate contributions.

1.3 2. Random Forest

Team Member: Saman

This notebook trains the data by Random Forest on the fully cleaned and standardized dataset produced in data_cleaning_merged.ipynb. The fitted model is evaluated on the 4,000-observation test set.

Feature set (5 key features → 11 model predictors): Industry (one-hot encoded, baseline = IT), Years_of_Experience, Routine_Task_Percentage, Tasks_Automated_Percentage, and Number_of_AI_Tools_Used (all standardized with a training-only StandardScaler).

Target: Layoff_Risk (0 = Low, 1 = Medium, 2 = High).

1.3.1 2.1 Training Data By Random Forest

Step 1: Model Initialization and Training (Fitting) **What We Did:** We initialized and fitted an ensemble `RandomForestClassifier` on the training partition ($N = 16,000$, 80%) using the revised 11-predictor specification

Hyperparameter Configuration: We parameterized the ensemble with `n_estimators=100` trees and `random_state=42` to guarantee exact replication across runs.

[59]:
```
# ============================================================================
# INITIALIZE AND FIT RANDOM FOREST CLASSIFIER
# ============================================================================

# Instantiate the Random Forest ensemble
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Fit directly on the training partition
rf_model.fit(X_train, y_train)
```

```
print("Random Forest Classifier fitted successfully!")
print("Model classes learned:", rf_model.classes_)
print(f"Number of decision trees constructed: {len(rf_model.estimators_)}")
print(f"Feature count evaluated at splits: {rf_model.n_features_in_}")
```

```
Random Forest Classifier fitted successfully!
Model classes learned: [0 1 2]
Number of decision trees constructed: 100
Feature count evaluated at splits: 11
```

Step 2: Generating Predictions on the Test Set (X_test) **What We Did:** We evaluated our fitted Random Forest ensemble on the 4,000 unseen test instances (X_{test}) by generating both hard class predictions and continuous probability distributions.

Inference Mechanism: Final discrete labels are assigned using majority consensus across the 100 decision trees, while continuous class probabilities represent the fraction of tree votes allocated to each risk tier [$P(\text{Low}), P(\text{Medium}), P(\text{High})$].

[60]:
```
# ============================================================================
# GENERATE PREDICTIONS ON TEST SET (RANDOM FOREST)
# ============================================================================

# Predict discrete class labels via ensemble majority voting
y_test_pred_rf = rf_model.predict(X_test)

# Predict class probability distributions (vote share across 100 trees)
y_test_proba_rf = rf_model.predict_proba(X_test)

# Sanity check: Inspect the first 5 testing predictions
comparison_sample_rf = pd.DataFrame({
    'True_Label': y_test.values[:5],
    'Predicted_Label': y_test_pred_rf[:5],
    'P(Low=0)': y_test_proba_rf[:5, 0].round(3),
    'P(Medium=1)': y_test_proba_rf[:5, 1].round(3),
    'P(High=2)': y_test_proba_rf[:5, 2].round(3)
})

print("--- RANDOM FOREST: FIRST 5 TEST PREDICTIONS ---")
display(comparison_sample_rf)
```

```
--- RANDOM FOREST: FIRST 5 TEST PREDICTIONS ---
```

	True_Label	Predicted_Label	P(Low=0)	P(Medium=1)	P(High=2)
0	0	0	0.99	0.01	0.00
1	2	2	0.00	0.00	1.00
2	2	2	0.00	0.02	0.98
3	2	2	0.00	0.05	0.95
4	1	1	0.36	0.62	0.02

Step 3: Computing Test Performance Metrics (Random Forest)

[61]:
```
# ============================================================================
# COMPUTE EVALUATION METRICS ON TESTING SET (RANDOM FOREST)
# ============================================================================

# ----------------------------------------------------------------------------
# Scalar Metrics (Accuracy and Multi-Class ROC-AUC)
# ----------------------------------------------------------------------------
test_acc_rf = accuracy_score(y_test, y_test_pred_rf)
test_auc_rf = roc_auc_score(y_test, y_test_proba_rf, multi_class='ovr',␣
 ↪average='macro')

print("=== RANDOM FOREST: TEST PERFORMANCE METRICS ===")
print(f"Test Accuracy: {test_acc_rf:.4f} ({test_acc_rf * 100:.2f}%)")
print(f"Test ROC-AUC (OvR Macro): {test_auc_rf:.4f}")

# ----------------------------------------------------------------------------
# Detailed Multi-Class Classification Report
# ----------------------------------------------------------------------------
print("\n--- CLASSIFICATION REPORT (PRECISION, RECALL, F1-SCORE) ---")
print(classification_report(y_test, y_test_pred_rf, target_names=['Low (0)',␣
 ↪'Medium (1)', 'High (2)'], digits=4))

# ----------------------------------------------------------------------------
# Confusion Matrix Visualization
# ----------------------------------------------------------------------------
cm_rf = confusion_matrix(y_test, y_test_pred_rf)
cm_labels = ['Low', 'Medium', 'High']

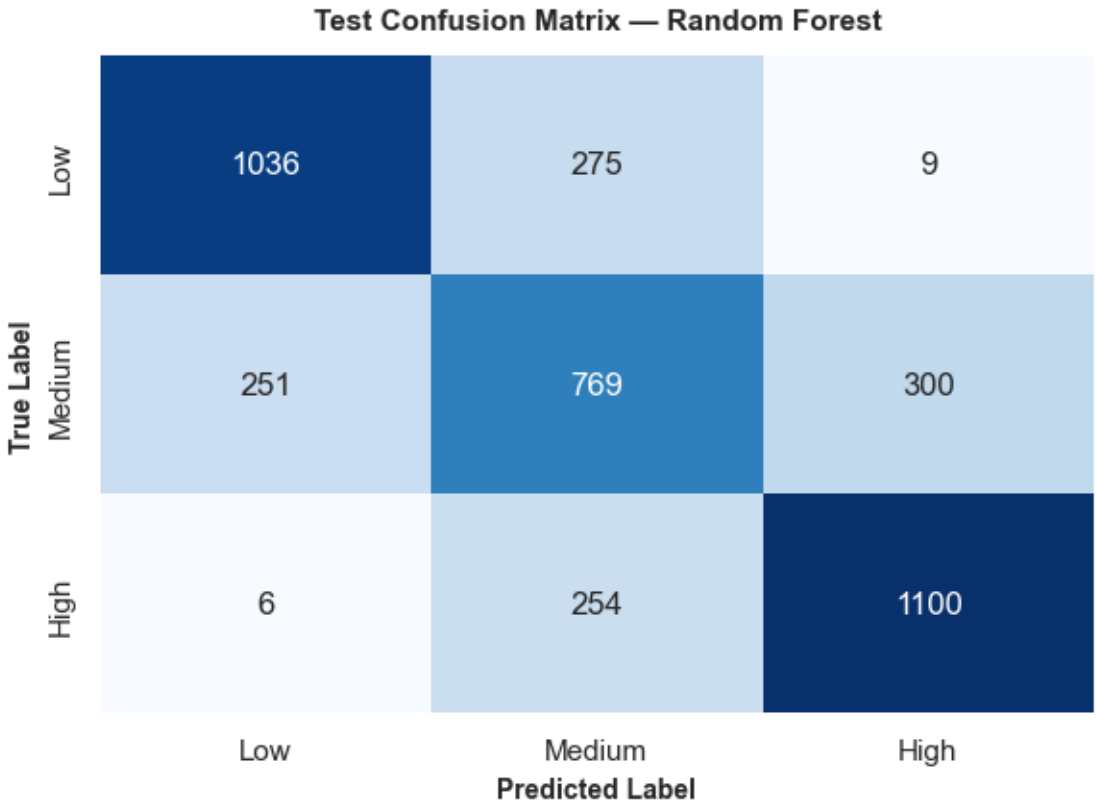
fig, ax = plt.subplots(figsize=(6, 4.5))
sns.heatmap(
    cm_rf,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=cm_labels,
    yticklabels=cm_labels,
    cbar=False,
    ax=ax
)
ax.set_title("Test Confusion Matrix - Random Forest", fontsize=11,␣
 ↪fontweight='bold', pad=10)
ax.set_xlabel("Predicted Label", fontsize=10, fontweight='bold')
ax.set_ylabel("True Label", fontsize=10, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
=== RANDOM FOREST: TEST PERFORMANCE METRICS ===
Test Accuracy: 0.7262 (72.62%)
Test ROC-AUC (OvR Macro): 0.8867

--- CLASSIFICATION REPORT (PRECISION, RECALL, F1-SCORE) ---
              precision    recall  f1-score   support

     Low (0)     0.8012    0.7848    0.7930      1320
  Medium (1)     0.5924    0.5826    0.5875      1320
    High (2)     0.7807    0.8088    0.7945      1360

    accuracy                         0.7262      4000
   macro avg     0.7248    0.7254    0.7250      4000
weighted avg     0.7254    0.7262    0.7257      4000
```



Evaluating the Random Forest ensemble on the 4,000 unseen test instances demonstrated strong non-linear predictive capability, achieving an overall test accuracy of 72.62% (2,905 of 4,000 correct) and a One-vs-Rest Macro ROC-AUC of 0.8867. This performance substantially outperforms the majority-class baseline (~34.00%), verifying that the 100-tree bagging ensemble effectively captures structural displacement signals from workload automation and industry predictors. Per-class metrics indicate robust discrimination at the operational extremes: Low Risk achieved a preci-

sion of 80.12% and recall of 78.48% (F1-score of 0.7930), while High Risk reached a precision of 78.07% and recall of 80.88% (F1-score of 0.7945). Medium Risk yielded an intermediate F1-score of 0.5875 (precision 59.24%, recall 58.26%), reflecting expected boundary ambiguities where transitional workloads share characteristics with neighboring tiers. The 3×3 confusion matrix further confirms strong ordinal stability with virtually no severe classification errors: out of 1,360 true High-risk workers, 1,100 were accurately identified, 254 were estimated as Medium risk, and only 6 (0.44%) were misclassified as Low risk. Similarly, among 1,320 true Low-risk instances, 1,036 were correctly identified, 275 were categorized as Medium risk, and only 9 (0.68%) were falsely escalated to High risk. Out of 1,320 true Medium-risk employees, 769 were correctly classified, while 251 were predicted as Low and 300 as High. Because errors occur almost entirely between adjacent risk thresholds rather than opposite extremes, the ensemble proves to be a well-calibrated and dependable benchmark for workforce risk modeling

[62]:
```
from sklearn.metrics import recall_score
import numpy as np
import pandas as pd

# Recall for the other categories (group 0) equals specificity.
specificity_rf = [
    recall_score(
        np.asarray(y_test) == label,
        np.asarray(y_test_pred_rf) == label,
        pos_label=0
    )
    for label in [0, 1, 2]
]

specificity_table_rf = pd.DataFrame(
    {"Specificity": specificity_rf},
    index=["Low", "Medium", "High"]
)

specificity_table_rf.loc["Macro average"] = np.mean(specificity_rf)

print("RANDOM FOREST - TEST SPECIFICITY")
display(specificity_table_rf.round(4))
```

```
RANDOM FOREST - TEST SPECIFICITY
```

	Specificity
Low	0.9041
Medium	0.8026
High	0.8830
Macro average	0.8632

Step 4: Gini Feature Importance Extraction and Visualization

[63]:
```
# ========================================================================
# EXTRACTING AND VISUALIZING FEATURE IMPORTANCE (RANDOM FOREST)
```

```
# ============================================================================

# Extract Gini importances and feature names
importances = rf_model.feature_importances_
feature_names = np.array(X_train.columns)

# Sort indices in descending order of importance
sort_idx = np.argsort(importances)[::-1]

# Construct the entire DataFrame in a single step (Zero warnings)
sorted_importances = importances[sort_idx]
sorted_pct = np.round(sorted_importances * 100, 2)
cum_pct = np.round(np.cumsum(sorted_pct), 2)

fi_df = pd.DataFrame({
    'Feature': feature_names[sort_idx],
    'Importance': sorted_importances,
    'Importance_Pct': sorted_pct,
    'Cumulative_Pct': cum_pct
})

print("=== RANDOM FOREST: FEATURE IMPORTANCE SUMMARY TABLE ===")
display(fi_df)

# Horizontal Bar Chart of Feature Importances
fig, ax = plt.subplots(figsize=(10, 6))

plot_fi = fi_df.sort_values(by='Importance', ascending=True)

bars = ax.barh(
    plot_fi['Feature'],
    plot_fi['Importance_Pct'],
    color='#3498db',
    edgecolor='black',
    linewidth=0.8
)

# Overlay percentage labels on the bars
for bar in bars:
    width = bar.get_width()
    ax.text(
        width + 0.5,
        bar.get_y() + bar.get_height() / 2,
        f'{width:.2f}%',
        va='center',
        ha='left',
        fontsize=9.5,
```

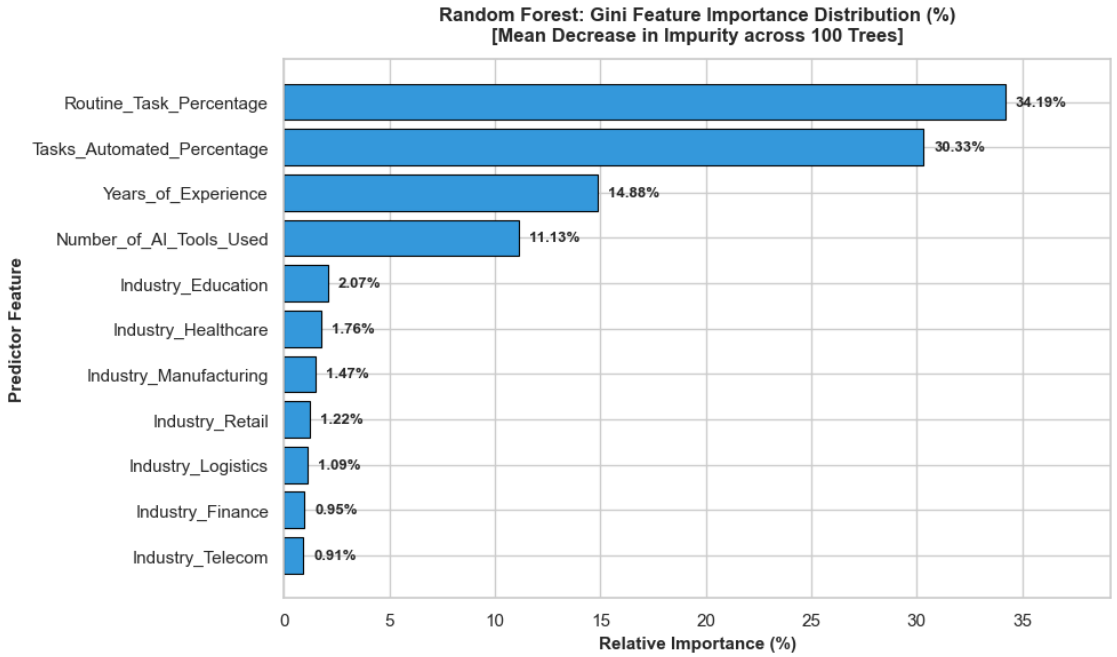
```
        fontweight='bold'
    )

ax.set_title("Random Forest: Gini Feature Importance Distribution (%)\n[Mean␣
 ↪Decrease in Impurity across 100 Trees]", fontsize=12, fontweight='bold',␣
 ↪pad=12)
ax.set_xlabel("Relative Importance (%)", fontsize=11, fontweight='bold')
ax.set_ylabel("Predictor Feature", fontsize=11, fontweight='bold')
ax.set_xlim(0, max(plot_fi['Importance_Pct']) + 5)

plt.tight_layout()
plt.show()
```

```
=== RANDOM FOREST: FEATURE IMPORTANCE SUMMARY TABLE ===
```

	Feature	Importance	Importance_Pct	Cumulative_Pct
0	Routine_Task_Percentage	0.341922	34.19	34.19
1	Tasks_Automated_Percentage	0.303271	30.33	64.52
2	Years_of_Experience	0.148768	14.88	79.40
3	Number_of_AI_Tools_Used	0.111330	11.13	90.53
4	Industry_Education	0.020658	2.07	92.60
5	Industry_Healthcare	0.017616	1.76	94.36
6	Industry_Manufacturing	0.014722	1.47	95.83
7	Industry_Retail	0.012214	1.22	97.05
8	Industry_Logistics	0.010872	1.09	98.14
9	Industry_Finance	0.009541	0.95	99.09
10	Industry_Telecom	0.009087	0.91	100.00



Evaluating the Gini feature importance across the 100-tree Random Forest ensemble indicates that employee displacement risk is overwhelmingly governed by task-level automation exposure rather than macroeconomic sector categories. The top two predictors—Routine Task Percentage (34.19%) and Tasks Automated Percentage (30.33%)—collectively account for 64.52% of all impurity reduction across the forest, establishing that the proportion of repetitiveness and direct AI displacement within a worker's day-to-day duties serves as the primary branching criterion. Demographic tenure (Years of Experience) ranks third at 14.88%, followed by technological interaction (Number of AI Tools Used) at 11.13%. In aggregate, these four continuous metrics constitute 90.53% of the model's total explanatory power, leaving only 9.47% distributed across all seven one-hot industry indicators. Among the sectoral features, Education (2.07%), Healthcare (1.76%), and Manufacturing (1.47%) provide modest split utility, while Retail (1.22%), Logistics (1.09%), Finance (0.95%), and Telecom (0.91%) contribute marginally to tree partitioning. These empirical weights confirm that while industry classification establishes subtle baseline adjustments, an individual employee's vulnerability to workforce reduction is predominantly determined by their operational workload profile and vulnerability to automated task substitution.

1.4 3. Gradient Boosting

Team Member: Mustafa

This notebook trains the ensemble boosting model (Gradient Boosting Classifier) on the fully cleaned and standardized dataset produced in `data_cleaning_merged.ipynb`. It evaluates the model strictly on the Testing set to avoid data leakage.

Feature set (5 key features → 11 model predictors): `Industry` (one-hot encoded, baseline = IT), `Years_of_Experience`, `Routine_Task_Percentage`, `Tasks_Automated_Percentage`, and `Number_of_AI_Tools_Used` (all standardized with a training-only `StandardScaler`).

Target: `Layoff_Risk` (0 = Low, 1 = Medium, 2 = High).

1.4.1 3.1 Model Initialization and Training

We use scikit-learn's `GradientBoostingClassifier` with its default hyperparameters (`n_estimators=100`, `learning_rate=0.1`, `max_depth=3`, `subsample=1.0`). Using defaults keeps this iteration a clean, untuned baseline so that any performance change relative to the previous demographic feature set can be attributed to the new features rather than to tuning.

[30]:
```
# ----------------------------------------------------------------------
# Fit Gradient Boosting Classifier
# ----------------------------------------------------------------------
# We initialize the Gradient Boosting Classifier ensemble algorithm.
# We set random_state=42 to ensure the exact same trees are built every time
# the code is run, enabling deterministic and reproducible results.
gbc = GradientBoostingClassifier(random_state=42)

# Fit the ensemble of decision trees strictly to the training partition.
gbc.fit(X_train, y_train)
```

```
print("Gradient Boosting training completed successfully!")
```

```
Gradient Boosting training completed successfully!
```

[31]:
```
# ------------------------------------------------------------------------
# Overfitting Check: Training vs. Testing Accuracy
# A large gap between the two would indicate the ensemble is memorizing the
# training set rather than learning generalizable patterns.
# ------------------------------------------------------------------------
train_acc = accuracy_score(y_train, gbc.predict(X_train))
test_acc = accuracy_score(y_test, gbc.predict(X_test))

print(f"Training Accuracy:   {train_acc:.4f} OR {train_acc * 100:.2f}%")
print(f"Testing Accuracy: {test_acc:.4f} OR {test_acc * 100:.2f}%")
print(f"Generalization Gap (Train - test): {train_acc - test_acc:+.4f}")
```

```
Training Accuracy:   0.7628 OR 76.28%
Testing Accuracy: 0.7495 OR 74.95%
Generalization Gap (Train - test): +0.0132
```

1.4.2 3.2 Testing Performance & Metrics

Because this is a multi-class classification problem (0: Low, 1: Medium, 2: High), we calculate: - **Overall Accuracy** - **Sensitivity (Recall)**: Calculated per class and as a macro average. - **Specificity (True Negative Rate)**: Calculated per class and as a macro average.

[32]:
```
# ------------------------------------------------------------------------
# Predict on Testing Set
# ------------------------------------------------------------------------
# Generate predicted class labels for the testing subset
y_pred = gbc.predict(X_test)

# Calculate the overall proportion of correct predictions across all classes
acc = accuracy_score(y_test, y_pred)
print(f"Testing Accuracy: {acc:.4f} OR {acc*100:.2f}%\n")

# Generate a confusion matrix to evaluate class-level true positives, false␣
 ↪positives, etc.
cm = confusion_matrix(y_test, y_pred)

# Define the categorical class labels for clean iteration and printing
classes = [0, 1, 2]
class_names = ['Low (0)', 'Medium (1)', 'High (2)']

print("--- Metric Evaluation ---")
sensitivities = []
specificities = []
```

```
# Iterate through each unique class to calculate One-vs-Rest metrics
for i, label in enumerate(classes):
    # True Positives: diagonal element for the current class
    tp = cm[i, i]

    # False Negatives: sum of actual class row minus true positives
    fn = cm[i, :].sum() - tp

    # False Positives: sum of predicted class column minus true positives
    fp = cm[:, i].sum() - tp

    # True Negatives: total observations minus (TP + FP + FN)
    tn = cm.sum() - (tp + fn + fp)

    # Calculate Sensitivity (Recall): True Positives / Actual Positives
    sens = tp / (tp + fn) if (tp + fn) > 0 else 0

    # Calculate Specificity (True Negative Rate): True Negatives / Actual␣
 ↪Negatives
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0

    # Append to lists so we can calculate macro averages later
    sensitivities.append(sens)
    specificities.append(spec)

    # Print the per-class results
    print(f"{class_names[i]}:")
    print(f"  - Sensitivity (Recall): {sens:.4f}")
    print(f"  - Specificity (TNR):    {spec:.4f}")

# Calculate and print the unweighted macro averages across all three classes
print("\n--- Macro Averages ---")
print(f"Macro Sensitivity: {np.mean(sensitivities):.4f}")
print(f"Macro Specificity: {np.mean(specificities):.4f}")
```

```
Testing Accuracy: 0.7495 OR 74.95%

--- Metric Evaluation ---
Low (0):
  - Sensitivity (Recall): 0.8000
  - Specificity (TNR):    0.9138
Medium (1):
  - Sensitivity (Recall): 0.6106
  - Specificity (TNR):    0.8209
High (2):
  - Sensitivity (Recall): 0.8353
  - Specificity (TNR):    0.8898
```

```
--- Macro Averages ---
Macro Sensitivity: 0.7486
Macro Specificity: 0.8748
```

[33]:
```python
# ------------------------------------------------------------------------
# Consolidated Per-Class Metrics Table & scikit-learn Classification Report
# Combines Sensitivity, Specificity, and Precision for every class, followed
# by the standard classification report (precision / recall / F1 / support).
# ------------------------------------------------------------------------
# Precision = TP / (TP + FP) -> diagonal of the confusion matrix divided by␣
 ↪predicted-class totals
precisions = np.diag(cm) / cm.sum(axis=0)

metrics_summary = pd.DataFrame({
    'Sensitivity (Recall)': sensitivities,
    'Specificity (TNR)': specificities,
    'Precision': precisions
}, index=class_names).round(4)
metrics_summary.loc['Macro Average'] = metrics_summary.mean().round(4)

print("--- PER-CLASS METRICS SUMMARY (TEST SET) ---")
display(metrics_summary)

print(f"\nOverall Testing Accuracy: {acc:.4f}")
print("\n--- SCIKIT-LEARN CLASSIFICATION REPORT ---")
print(classification_report(y_test, y_pred, target_names=class_names, digits=4))
```

```
--- PER-CLASS METRICS SUMMARY (TEST SET) ---
```

	Sensitivity (Recall)	Specificity (TNR)	Precision
Low (0)	0.8000	0.9138	0.8205
Medium (1)	0.6106	0.8209	0.6267
High (2)	0.8353	0.8898	0.7961
Macro Average	0.7486	0.8748	0.7478

```
Overall Testing Accuracy: 0.7495

--- SCIKIT-LEARN CLASSIFICATION REPORT ---
              precision    recall  f1-score   support

     Low (0)     0.8205    0.8000    0.8101      1320
  Medium (1)     0.6267    0.6106    0.6186      1320
    High (2)     0.7961    0.8353    0.8152      1360

    accuracy                         0.7495      4000
   macro avg     0.7478    0.7486    0.7480      4000
weighted avg     0.7483    0.7495    0.7486      4000
```

Empirical Takeaway — Boosting Ensemble Performance: With the workload- and automation-centric feature set, Gradient Boosting reaches a test accuracy of 74.95%**. Macro sensitivity reaches** 0.7486** and macro specificity reaches **0.8748**. The model separates the extreme tiers cleanly: **High Risk** achieves the ensemble's highest recall (**0.8353**, F1 0.8152, Precision 0.7961), while **Low Risk** attains the strongest specificity (**0.9138**, F1 0.8101, Precision **0.8205**). **Medium Risk** remains the most challenging tier (sensitivity **0.6106**, precision **0.6267**, F1 0.6186), which is expected because the transitional cohort sits between two well-separated polar classes and its feature distributions naturally overlap with both. The minimal generalization gap demonstrates robust regularization with no material overfitting, establishing Gradient Boosting as the most sensitive detector of high-risk displacement in the pipeline. The train/test gap is small (**76.28% vs. 74.95%, +0.0132 pts**), indicating a well-generalized model with no material overfitting.

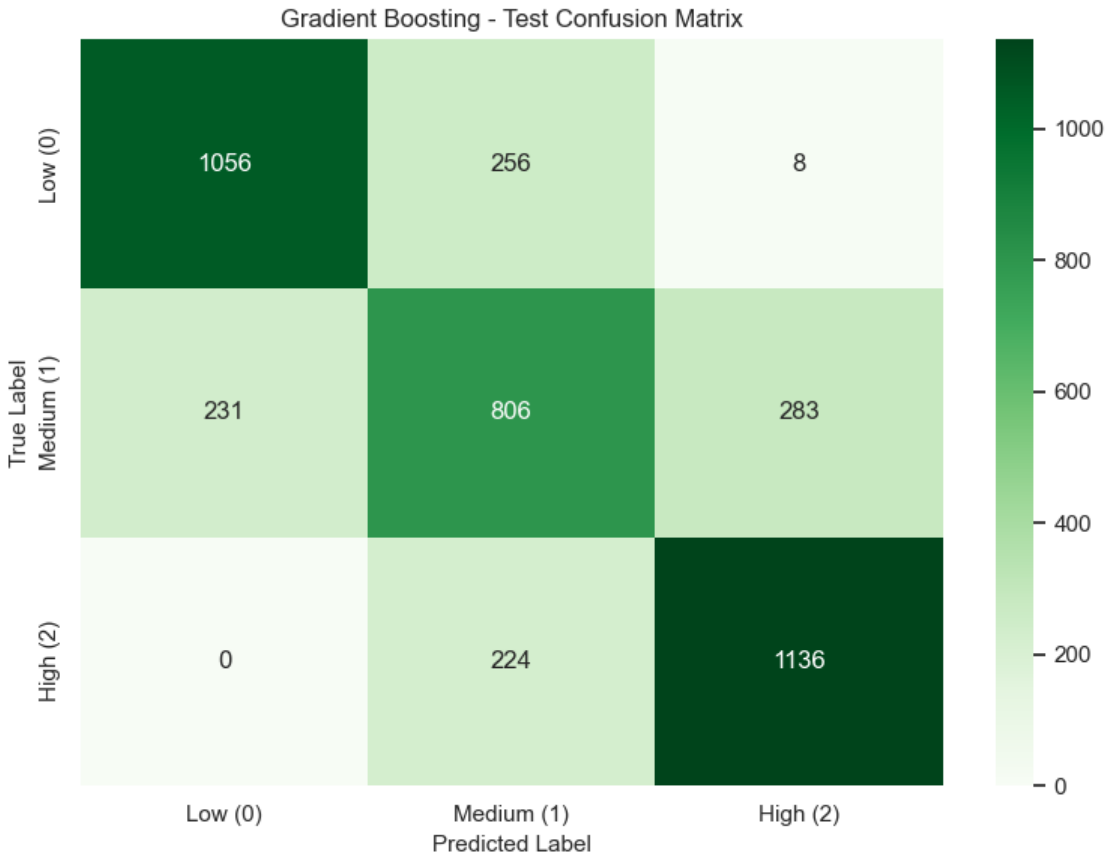
[34]:
```
# ------------------------------------------------------------------------
# Visualizing the Confusion Matrix
# ------------------------------------------------------------------------
# Create a matplotlib figure to host the seaborn heatmap
plt.figure(figsize=(8, 6))

# Draw the heatmap, annotating cells with raw counts (fmt='d')
sns.heatmap(cm, annot=True, fmt="d", cmap="Greens",
            xticklabels=class_names, yticklabels=class_names)

# Add axis labels and title for academic clarity
plt.title("Gradient Boosting - Test Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

# Use tight_layout to prevent clipping of labels during image export
plt.tight_layout()

# Display the figure inline
plt.show()
```



Empirical Takeaway — Gradient Boosting Confusion Matrix Analysis:The 3×3 confusion matrix confirms strong ordinal consistency and class separability for the Gradient Boosting model across the 4,000 test observations. Out of 1,360 true High-risk workers, the ensemble correctly identifies 1,136 (83.53%) and redirects 224 to the adjacent Medium tier, while committing zero catastrophic misclassifications into the Low Risk category. Similarly, of the 1,320 true Low-risk instances, 1,056 (80.00%) are correctly flagged, 256 are placed in Medium risk, and only 8 (0.61%) are falsely escalated to High Risk. Misclassifications occur almost exclusively across adjacent thresholds within the intermediate Medium Risk cohort (where 231 are predicted as Low and 283 as High), demonstrating that error distributions reflect natural transition overlap rather than structural model instability.

1.4.3 3.3 Feature Importance

Gradient Boosting exposes impurity-based feature importances, aggregating each predictor's total contribution to loss reduction across all trees. This reveals which of the 11 predictors actually drive the model's layoff-risk predictions.

[35]:
```
# ------------------------------------------------------------------------
# Feature Importance (Impurity-Based)
```

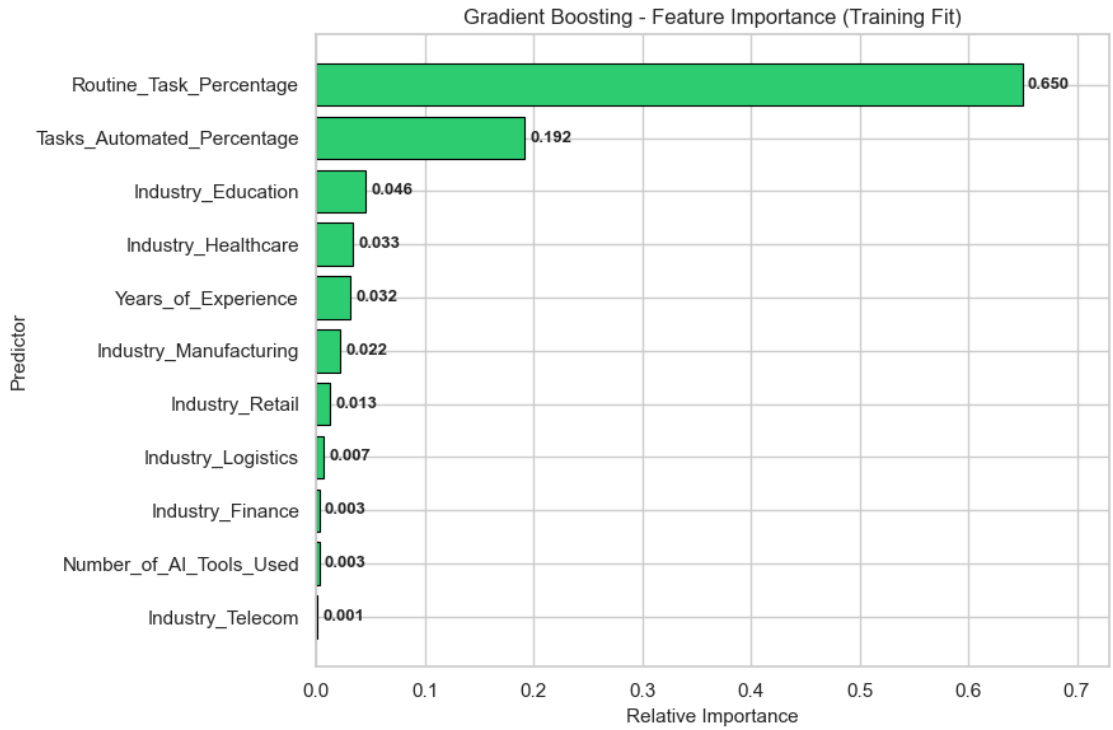
```
# Rank all 11 predictors by their relative contribution to the ensemble.
# ------------------------------------------------------------------------
importances = pd.Series(gbc.feature_importances_, index=X_train.columns).
 ↪sort_values(ascending=True)

plt.figure(figsize=(9, 6))
bars = plt.barh(importances.index, importances.values, color='#2ecc71',␣
 ↪edgecolor='black', linewidth=0.8)

# Annotate each bar with its importance value
for bar, val in zip(bars, importances.values):
    plt.text(val + 0.005, bar.get_y() + bar.get_height() / 2, f"{val:.3f}",␣
 ↪va='center', fontsize=9.5, fontweight='bold')

plt.title("Gradient Boosting - Feature Importance (Training Fit)")
plt.xlabel("Relative Importance")
plt.ylabel("Predictor")
plt.xlim(0, importances.max() + 0.08)
plt.tight_layout()
plt.show()

print("--- FEATURE IMPORTANCE TABLE (DESCENDING) ---")
display(importances.sort_values(ascending=False).round(4).
 ↪to_frame('Importance'))
```



```
--- FEATURE IMPORTANCE TABLE (DESCENDING) ---

                            Importance
Routine_Task_Percentage         0.6498
Tasks_Automated_Percentage      0.1920
Industry_Education              0.0455
Industry_Healthcare             0.0334
Years_of_Experience             0.0317
Industry_Manufacturing          0.0217
Industry_Retail                 0.0127
Industry_Logistics              0.0069
Industry_Finance                0.0027
Number_of_AI_Tools_Used         0.0025
Industry_Telecom                0.0009
```

> **Empirical Takeaway — What Drives the Predictions:** Importance is heavily concentrated in two predictors: **Routine_Task_Percentage (0.6498)** and **Tasks_Automated_Percentage (0.1920)** together account for roughly 84.18% of the ensemble's total importance, mirroring the strong tier separation seen in the EDA. Industry indicators contribute modestly (e.g., Education 0.0455, Healthcare 0.0334, and Manufacturing 0.0217), while Years_of_Experience adds a small secondary effect (0.0317). Notably, Number_of_AI_Tools_Used receives minimal importance (0.0025) despite its strong marginal relationship with risk in the EDA; this is explained by its collinearity with the two dominant predictors ($r = 0.66$ with routine tasks, $r = 0.59$ with tasks automated), offering little incremental information once those core workload features are split upon. Caveat: Impurity-based importance can be shared arbitrarily among correlated predictors, so these values should be interpreted as indicative feature reliance rather than causal mechanisms.

1.5 4. Gaussian Naïve Bayes

Team Member: Mustafa

This notebook trains the probabilistic baseline model (Gaussian Naïve Bayes) on the fully cleaned and standardized dataset produced in `data_cleaning_merged.ipynb`. It evaluates the model strictly on the Testing set to avoid data leakage.

Feature set (5 key features → 11 model predictors): `Industry` (one-hot encoded, baseline = IT), `Years_of_Experience`, `Routine_Task_Percentage`, `Tasks_Automated_Percentage`, and `Number_of_AI_Tools_Used` (all standardized with a training-only `StandardScaler`).

Target: `Layoff_Risk` (0 = Low, 1 = Medium, 2 = High).

1.5.1 4.1 Model Initialization and Training

Gaussian Naïve Bayes models each predictor within each class as a normal distribution and combines them under the assumption of **conditional independence**. Two properties of our feature set are worth noting up front: (1) `Routine_Task_Percentage` and `Tasks_Automated_Percentage` are strongly correlated ($r = 0.89$), which violates the independence assumption, and (2) the 7

`Industry_*` one-hot indicators are binary rather than Gaussian. Both can limit the model, so this notebook serves as a probabilistic baseline against which Gradient Boosting is compared.

[36]:
```
# ------------------------------------------------------------------------
# Fit Gaussian Naïve Bayes
# ------------------------------------------------------------------------
# Initialize the Gaussian Naïve Bayes model.
# This model assumes features follow a normal distribution and are␣
 ↪conditionally independent.
gnb = GaussianNB()

# Fit the model strictly to the training data. This calculates the prior␣
 ↪probabilities
# and class-conditional likelihoods for each feature based ONLY on X_train and␣
 ↪y_train.
gnb.fit(X_train, y_train)

print("Gaussian Naïve Bayes training completed successfully!")
```

```
Gaussian Naïve Bayes training completed successfully!
```

[37]:
```
# ------------------------------------------------------------------------
# Overfitting Check: Training vs. Test Accuracy
# Naïve Bayes has very low capacity, so train and test accuracy should be close.
# ------------------------------------------------------------------------
train_acc = accuracy_score(y_train, gnb.predict(X_train))
test_acc = accuracy_score(y_test, gnb.predict(X_test))

print(f"Training Accuracy:   {train_acc:.4f} OR {train_acc * 100:.2f}%")
print(f"Testing Accuracy: {test_acc:.4f} OR {test_acc * 100:.2f}%")
print(f"Generalization Gap (Train - test): {train_acc - test_acc:+.4f}")

# Learned class prior probabilities P(class), estimated from X_train/y_train␣
 ↪only
print("\n--- LEARNED CLASS PRIORS ---")
display(pd.Series(gnb.class_prior_, index=['Low (0)', 'Medium (1)', 'High␣
 ↪(2)'], name='Prior').round(4).to_frame())
```

```
Training Accuracy:   0.7231 OR 72.31%
Testing Accuracy: 0.7335 OR 73.35%
Generalization Gap (Train - test): -0.0104

--- LEARNED CLASS PRIORS ---
```

	Prior
Low (0)	0.3301
Medium (1)	0.3301
High (2)	0.3398

1.5.2 4.2 Testing Performance & Metrics

Because this is a multi-class classification problem (0: Low, 1: Medium, 2: High), we calculate: - **Overall Accuracy** - **Sensitivity (Recall)**: Calculated per class and as a macro average. - **Specificity (True Negative Rate)**: Calculated per class and as a macro average.

[38]:
```
# ------------------------------------------------------------------------
# Predict on Testing Set
# ------------------------------------------------------------------------
# Generate predicted class labels for the test subset
y_pred = gnb.predict(X_test)

# Calculate the overall proportion of correct predictions across all classes
acc = accuracy_score(y_test, y_pred)
print(f"Testing Accuracy: {acc:.4f}\n")

# Generate a confusion matrix to evaluate class-level true positives, false␣
 ↪positives, etc.
cm = confusion_matrix(y_test, y_pred)

# Define the categorical class labels for clean iteration and printing
classes = [0, 1, 2]
class_names = ['Low (0)', 'Medium (1)', 'High (2)']

print("--- Metric Evaluation ---")
sensitivities = []
specificities = []

# Iterate through each unique class to calculate One-vs-Rest metrics
for i, label in enumerate(classes):
    # True Positives: diagonal element for the current class
    tp = cm[i, i]

    # False Negatives: sum of actual class row minus true positives
    fn = cm[i, :].sum() - tp

    # False Positives: sum of predicted class column minus true positives
    fp = cm[:, i].sum() - tp

    # True Negatives: total observations minus (TP + FP + FN)
    tn = cm.sum() - (tp + fn + fp)

    # Calculate Sensitivity (Recall): True Positives / Actual Positives
    sens = tp / (tp + fn) if (tp + fn) > 0 else 0

    # Calculate Specificity (True Negative Rate): True Negatives / Actual␣
 ↪Negatives
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0
```

```
    # Append to lists so we can calculate macro averages later
    sensitivities.append(sens)
    specificities.append(spec)

    # Print the per-class results
    print(f"{class_names[i]}:")
    print(f"  - Sensitivity (Recall): {sens:.4f}")
    print(f"  - Specificity (TNR):    {spec:.4f}")

# Calculate and print the unweighted macro averages across all three classes
print("\n--- Macro Averages ---")
print(f"Macro Sensitivity: {np.mean(sensitivities):.4f}")
print(f"Macro Specificity: {np.mean(specificities):.4f}")
```

```
Testing Accuracy: 0.7335

--- Metric Evaluation ---
Low (0):
  - Sensitivity (Recall): 0.8659
  - Specificity (TNR):    0.8616
Medium (1):
  - Sensitivity (Recall): 0.5311
  - Specificity (TNR):    0.8388
High (2):
  - Sensitivity (Recall): 0.8015
  - Specificity (TNR):    0.9004

--- Macro Averages ---
Macro Sensitivity: 0.7328
Macro Specificity: 0.8669
```

[39]:
```
# ------------------------------------------------------------------------
# Consolidated Per-Class Metrics Table & scikit-learn Classification Report
# Combines Sensitivity, Specificity, and Precision for every class, followed
# by the standard classification report (precision / recall / F1 / support).
# ------------------------------------------------------------------------
# Precision = TP / (TP + FP) -> diagonal of the confusion matrix divided by␣
 ↪predicted-class totals
precisions = np.diag(cm) / cm.sum(axis=0)

metrics_summary = pd.DataFrame({
    'Sensitivity (Recall)': sensitivities,
    'Specificity (TNR)': specificities,
    'Precision': precisions
}, index=class_names).round(4)
metrics_summary.loc['Macro Average'] = metrics_summary.mean().round(4)
```

```
print("--- PER-CLASS METRICS SUMMARY (TEST SET) ---")
display(metrics_summary)

print(f"\nOverall Testing Accuracy: {acc:.4f}")
print("\n--- SCIKIT-LEARN CLASSIFICATION REPORT ---")
print(classification_report(y_test, y_pred, target_names=class_names, digits=4))
```

```
--- PER-CLASS METRICS SUMMARY (TEST SET) ---
```

	Sensitivity (Recall)	Specificity (TNR)	Precision
Low (0)	0.8659	0.8616	0.7550
Medium (1)	0.5311	0.8388	0.6187
High (2)	0.8015	0.9004	0.8056
Macro Average	0.7328	0.8669	0.7264

```
Overall Testing Accuracy: 0.7335

--- SCIKIT-LEARN CLASSIFICATION REPORT ---
              precision    recall  f1-score   support

     Low (0)     0.7550    0.8659    0.8066      1320
  Medium (1)     0.6187    0.5311    0.5715      1320
    High (2)     0.8056    0.8015    0.8035      1360

    accuracy                         0.7335      4000
   macro avg     0.7264    0.7328    0.7272      4000
weighted avg     0.7272    0.7335    0.7280      4000
```

Empirical Takeaway — Probabilistic Performance: Gaussian Naïve Bayes achieves a **test accuracy of 73.35%**, with macro sensitivity **0.7328** and macro specificity **0.8669**. Performance is asymmetric across classes: **Low Risk** has the highest recall (**0.8659**, F1 0.8066) and High Risk has the highest precision **(0.8056)**, specificity **(0.9004)**, and strong recall (**0.8015**, F1 0.8035), while **Medium Risk** is the clear weak point (sensitivity only **0.5311**, precision **0.6187**, F1 0.5715) because the model frequently assigns middle-tier workers to the adjacent extreme tiers. Training and test accuracy are nearly identical (**72.31% vs. 73.35%**, generalization gap of **-1.04 pts**), confirming the low-capacity model neither over- nor under-fits relative to its assumptions. It trails Gradient Boosting by 1.60 percentage points (**73.35% vs. 74.95%**), consistent with its conditional-independence assumption being violated by the strongly correlated routine-task and automation features.

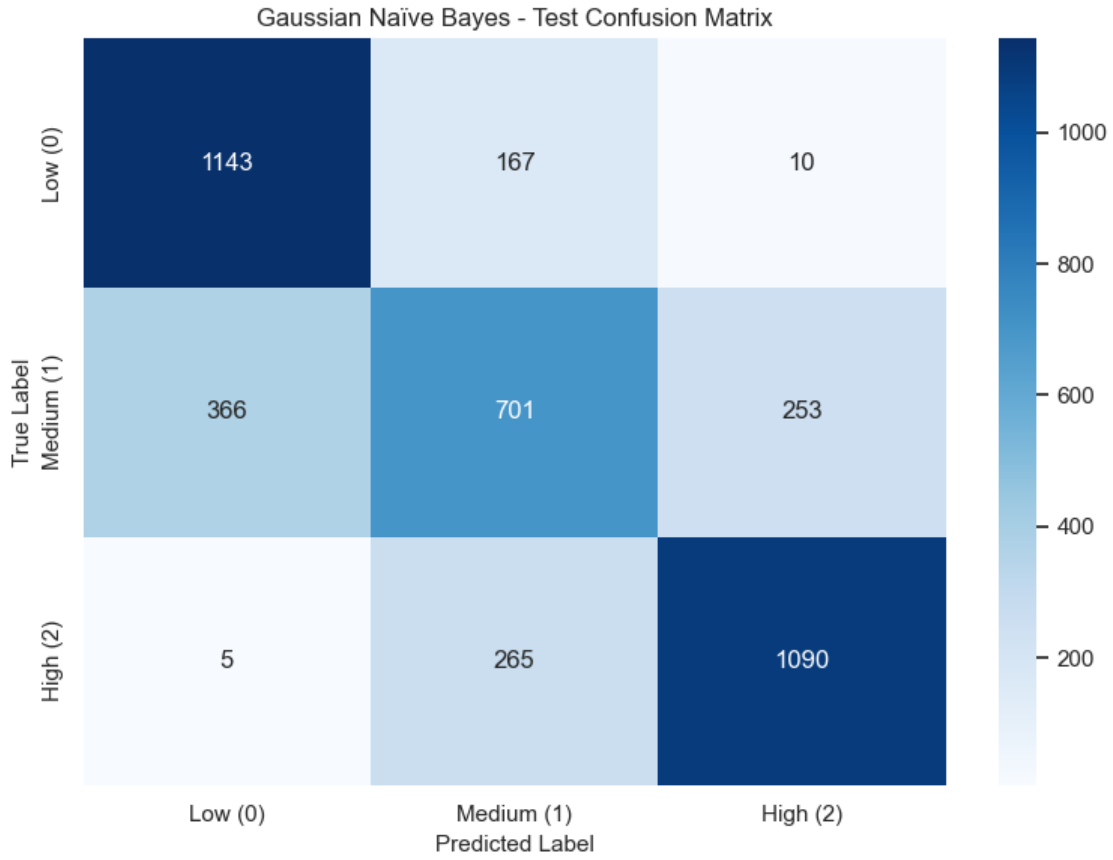
[40]:
```
# ------------------------------------------------------------------------
# Visualizing the Confusion Matrix
# ------------------------------------------------------------------------
# Create a matplotlib figure to host the seaborn heatmap
plt.figure(figsize=(8, 6))
```

```
# Draw the heatmap, annotating cells with raw counts (fmt='d')
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)

# Add axis labels and title for academic clarity
plt.title("Gaussian Naïve Bayes - Test Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

# Use tight_layout to prevent clipping of labels during image export
plt.tight_layout()

# Display the figure inline
plt.show()
```



Empirical Takeaway — Gaussian Naïve Bayes Confusion Matrix Dynamics: The confusion matrix reflects both the generative strengths and structural liabilities of Gaussian Naïve Bayes across the 4,000 test cases. The model preserves strong ordinal safety at the extremes: catastrophic cross-tier errors are virtually absent, with only

5 true High-risk workers (**0.37%**) misclassified as Low Risk and 10 Low-risk workers (**0.76**) flagged as High. Its primary limitation appears in the transitional Medium Risk tier, where it correctly captures only 701 of 1,320 instances (**53.11** recall). The remaining 619 middle-tier cases disperse into adjacent categories (366 into Low, 253 into High). This dispersion illustrates the classic limitation of factorized likelihoods: unable to parameterize feature covariance, the model over-amplifies tail probabilities when continuous feature values sit between distributions, squeezing intermediate observations outward into the polar classes.

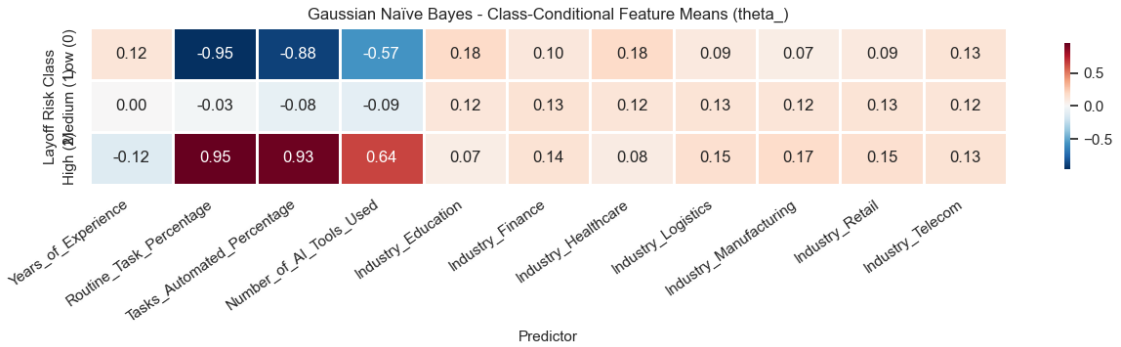
1.5.3 4.3 Learned Class-Conditional Feature Means

Gaussian Naïve Bayes stores, for every class, the mean of each predictor (in standardized units for the numeric features, and as the class-conditional proportion for the one-hot industry indicators). Comparing these means shows which features the model uses to distinguish the risk tiers.

[41]:
```
# ------------------------------------------------------------------------
# Class-Conditional Feature Means (theta_)
# Rows = classes, columns = predictors. Numeric features are in z-score units.
# ------------------------------------------------------------------------
theta_df = pd.DataFrame(
    gnb.theta_,
    index=['Low (0)', 'Medium (1)', 'High (2)'],
    columns=X_train.columns
)

plt.figure(figsize=(13, 3.8))
sns.heatmap(theta_df, annot=True, fmt=".2f", cmap="RdBu_r", center=0,␣
 ↪linewidths=0.8, cbar_kws={"shrink": 0.8})
plt.title("Gaussian Naïve Bayes - Class-Conditional Feature Means (theta_)")
plt.xlabel("Predictor")
plt.ylabel("Layoff Risk Class")
plt.xticks(rotation=35, ha='right')
plt.tight_layout()
plt.show()

print("--- CLASS-CONDITIONAL MEANS TABLE ---")
display(theta_df.round(3))
```



```
--- CLASS-CONDITIONAL MEANS TABLE ---
```

	Years_of_Experience	Routine_Task_Percentage	Tasks_Automated_Percentage	Number_of_AI_Tools_Used	Industry_Education	Industry_Finance	Industry_Healthcare	Industry_Logistics	Industry_Manufacturing	Industry_Retail	Industry_Telecom
Low (0)	0.124	-0.951	-0.885	-0.572	0.181	0.104	0.180	0.094	0.074	0.091	0.129
Medium (1)	0.004	-0.027	-0.077	-0.091	0.116	0.135	0.125	0.128	0.120	0.126	0.124
High (2)	-0.124	0.950	0.935	0.643	0.072	0.137	0.079	0.150	0.175	0.151	0.135

Empirical Takeaway — How Naïve Bayes Separates the Tiers: The class-conditional means show a near-symmetric ordering on the three workload features. Standardized `Routine_Task_Percentage` averages **−0.951** for Low Risk, **−0.027** for Medium, and **+0.950** for High; `Tasks_Automated_Percentage` follows (**−0.885 / −0.077 / +0.935**), and `Number_of_AI_Tools_Used` follows (**−0.572 / −0.091 / +0.643**). `Years_of_Experience` shows only a small gradient (**+0.124 / +0.004 / −0.124**), confirming its secondary role. Industry indicators differ across classes mainly for Education and Healthcare (e.g., 18.1% and 18.0% of Low Risk workers vs. 7.2% and 7.9% of High Risk) and Manufacturing (7.4% Low vs. 17.5% High). Because the Medium class mean sits almost exactly at the global mean (≈ 0) on every numeric feature, a symmetric Gaussian has little to distinguish it from the tails of its neighbours—which directly explains its low recall of 0.5311.

[ ]: